claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_7049e1ed-b93\agent-ab614e21a2ab5d791.jsonl`
- SHA-256: `7b7f39a2b8772f689663f7c1f86efa382ba10c01c05c196233c6c4a521ef7d36`
- Source modified: `2026-07-02T04:58:14+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_7049e1ed-b93`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T04:45:39.416Z)

You are a senior engineer producing ONE isolated PR. Today is 2026-07-02.
REPO: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (a shared checkout ? follow the safety rules exactly).
AUDIT FINDINGS YOU ARE FIXING: R1-02 (#340 Gemini re-bill) ? Grep C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md for these IDs and read the full evidence + verifier notes before coding.
TASK BRIEF:
Fix issue #340 in C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (USER-APPROVED behavior change, merge-on-green authorized): in the process-job execution path (backend/main.py ~:1172 _execute_processing ? re-verify exact location), when the requested video_id already exists with status='processed' and stored play_segments, short-circuit: return the stored segments as the job result WITHOUT re-running the segmenter (no Gemini call, no compute dispatch). Add ProcessRequest.force (default false) to deliberately reprocess; thread it through the async job boundary (jobs.params JSON or the existing columns ? check how 'compute' was threaded in schema v8 and mirror that; if a schema change is needed check PRAGMA user_version FIRST and take the next version). Log a clear '[IDEMPOTENT]' line when short-circuiting. Tests: short-circuit hit (no segmenter invocation ? assert via mock), force=true bypass, unprocessed video unaffected, boundary threading. Reference issues #340 + #330 in the PR body (closes #340).

BRANCHING SAFETY (mandatory ? shared checkout, sibling sessions exist):
1. In the MAIN repo checkout run: git fetch origin
2. Create an ISOLATED WORKTREE: git worktree add "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-<shortname>" -b <branch-name> origin/master   (branch explicitly FROM origin/master ? never from the current local branch; use origin/main for repos whose default is main)
3. Do ALL work inside the worktree. Before committing: git branch --show-current + git log --oneline -1 must show YOUR branch on the origin-master base.
4. Stage EXPLICIT paths only (git add <file> <file>) ? NEVER git add -A / . / -u
5. Before opening the PR: git log --oneline origin/master..HEAD must list ONLY your commits.
COMMIT/PR CONVENTIONS:
- Commit message: conventional style matching the repo's history; end the message with a blank line then: Co-Authored-By: Claude Fable 5 <noreply@anthropic.com>
- PR body: what/why, a "## Verification" section stating exactly what you ran locally and what CI covers, and end with: ? Generated with [Claude Code](https://claude.com/claude-code)
- Do NOT merge the PR. Do NOT remove the worktree (a fixer may need it). Return the PR number, branch, worktree path.
QUALITY BAR:
- Read the repo's CLAUDE.md first; for the engine also docs/CODE_MAP.md §0 (file placement) before adding any new file.
- Add/extend tests for every behavior change. Run the targeted tests you touched (python -m pytest tests/<relevant> -q) + ruff check on changed files; if mypy config exists, run mypy on

touched modules. Full suite is CI's job ? do not run it.
- Default-safe: new knobs default OFF / behavior-preserving unless the task explicitly says otherwise.
- The full audit evidence is in C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md ? Grep it for your finding IDs and READ the verifier notes; they contain corrections (exact file:line, scoping traps) you MUST honor.

If the brief's premise turns out false against current code (something already fixed, file moved), adapt honestly and say so in notes ? never force a pointless diff. If you genuinely cannot complete (env broken, auth missing), open NO PR and explain in notes with pr_number=0.

2. user (2026-07-02T04:45:47.111Z)

11:[Omitted long matching line]
44:[Omitted long matching line]
47:[Omitted long matching line]
61:### R1-02 [P1/M] critical-fix ? badminton-highlight-indexer
63:- anchor: `backend/main.py:1172` · tracked in: Khelsutra/badminton-highlight-indexer#340 (also #330) · finder: engine:correctness
64:- evidence: backend/main.py:1171-1172 computes video_id and `was_reingest = db.get_video(video_id) is not None`, but was_reingest is only used for the report/destroy-after-confirm; there is NO reuse/skip branch ? the segmenter always runs (main.py:1251-1255). Verified against gh: issue #340 OPEN ('re-runs the full (paid) segmenter anyway'), plus #330 (MD5 segmentation reuse) OPEN.
65:- why it matters: A page refresh or SPA 'Try again' on an already-indexed video (same MD5) re-bills Gemini (~$0.10/8-min match) and burns a multi-minute compute run for zero new information. Live-triggered once in the alpha (2026-06-22, per #340).
67:[Omitted long matching line]
68:[Omitted long matching line]
154:[Omitted long matching line]
561:- [P0] badminton-highlight-indexer (Khelsutra): #340/#330: /api/process still re-bills Gemini on re-submit of an already-processed video ? no idempotency short-circuit (live P0 cost bug)

3. user (2026-07-02T04:45:48.345Z)

CLAUDE.md
README.md
__pycache__
artifacts
backend
backend.db
config.json
deploy
docs
eval_baselines
indexer.db
mypy.ini
output
pyproject.toml
pytest.ini
rally_index.db
requirements.txt
ruff.toml
run_all_tests.py
run_eval.py
run_regression.py
scratch
scripts
tests
tools
training

4. user (2026-07-02T04:45:55.727Z)

```
40      ### cross:repo
41      Workspace of 5 git repos (all just pulled to origin master/main, zero extra worktrees)
plus 2 non-git data dirs ("Annotation Setup" 4.9G raw video/labels ? the n=6 golden labels ARE
versioned inside sports-data-collector's committed data/sports_metadata.db per commit 9e6f204,
so the dir is raw-media overflow; khelsutra-screencast 107M e2e artifacts). The engine<->SPA
runtime API contract is coherent in source: every SPA-called endpoint exists in backend/main.py
+ backend/api/uploads.py, and the new fps/resolution fields landed on both sides (engine
main.py:516-517 via #413; SPA types.ts:44-45 + lib/video.ts:35-36 via #140). The real cross-repo
drift is the COMMITTED vendored SPA bundle (backend/ui @ khelsutra ed33a515, 2026-06-22) now 10
web/ commits behind, with a skew guard that by design only catches engine-side API_VERSION
bumps. CI maturity is very uneven (engine: 4 guard lanes; khelsutra: excellent incl. coverage
floor + layout/a11y; sdc: pytest-only; rally-annotator: ok; obsreport: none), and docs in both
engine and khelsutra still describe the DO droplet deploy as LIVE a week after its 2026-06-25
retirement. Trees are clean except sports-data-collector (uncommitted .gitignore WIP + a stale
full nested self-clone), and 20 merged branches linger across remotes/locals.
42
43      ### engine:issues-triage
44      Engine repo is active and healthy on master (last merge #413 on /api/videos; CI trusted
green; ~1500 tests, coverage floor 70 enforced in both CI lanes). 13 issues open (not 14) plus
one PR. Triage found one live P0 cost bug (#340: /api/process re-runs the paid Gemini segmenter
for an already-processed video_id ? verified no dedup short-circuit exists in current code), the
D1 god-module-split PR #390 now CONFLICTING and 6 days stale while main.py grew to 2198 lines,
and several issues with partially-stale premises (#388 half-done since PR #234, #331/#326/#328
still citing the retired DO droplet, #175 referencing a design doc that does not exist in the
workspace). No fully fixed-but-open issues; the debt level is moderate and mostly well-tracked,
but the issue text drift means several tickets need scope-correcting comments before anyone acts
on them.
45
46      ### engine:perf-resource
47      Engine repo at origin/master e410136 (clean tree). Performance/resource-lifecycle
posture is mixed: the deliberately-built safeguards are genuinely wired and healthy ? SQLite
goes through a single choke point with WAL + busy_timeout=30000 + connect timeout=30s
(backend/infrastructure/database.py:48-57; the only sqlite3.connect in backend/, so the engine
does NOT have sports-data-collector's contention gap), keep_frames defaults False with post-
success frame/stage deletion in all three detector runners (config/models.py:61,98;
wasb_runner.py:210; tracknet_runner.py:225; native_wasb_runner.py:315), Gemini chunk files are
removed after analysis (segmenters/gemini.py:289,375-377), the stitcher clips live in a
TemporaryDirectory (compiler.py:419), and no unwaited Popen exists (no zombie risk). The debt is
concentrated where blocking work meets the max_workers=1 job executor: every serving-path ffmpeg
call runs without a timeout (extraction, audio, trim, per-clip encode, concat) so one hung
ffmpeg wedges all job processing; ExecutionPolicy/run_bounded protects only the Gemini generate
call. All four known perf/lifecycle issues (#349 single-threaded streaming decode, #331 CPU-
bound phases, #330 MD5 reuse, #301 scheduled retention sweep) are verified still open and
unimplemented in code, and one untracked leak was found: remote-input scratch copies (rally-
input-*/rally-worker-* mkdtemp dirs) are never deleted and are invisible to
tools/cleanup_caches.py.
48
49
50      ## Confirmed findings ? round 1 (43)
51
52      ### R1-01 [P1/M] tech-debt ? badminton-highlight-indexer
53      **PR #390 (D1/P10 main.py split) is stale and CONFLICTING while main.py grew 52% since
it was cut**
54      - anchor: `backend/main.py:1` · tracked in: PR #390;
docs/CODE_CLEANUP_AUDIT_2026-06-24.md §6 row P10 · finder: engine:audit-doc-reconcile
55      - evidence: gh pr view 390: state=OPEN, mergeable=CONFLICTING, mergeStateStatus=DIRTY,
updatedAt=2026-06-26T14:37Z, 3 files (backend/main.py, backend/orchestration.py, mypy.ini).
Since then >=8 PRs modified backend/main.py (#400, #405-#409, #412, #413 per git log
--since=2026-06-25 -- backend/main.py). wc -l backend/main.py = 2198 vs the audit doc's '1448
lines' claim (docs/CODE_CLEANUP_AUDIT_2026-06-24.md:279); 18 @app. routes and 45 print() calls
now live in it; backend.main still mypy-quarantined (mypy.ini:15). No backend/orchestration.py
exists on master.
56      - why it matters: This is the last HIGH-severity item of the cleanup audit (P10, the
```

only non-Phase-3 ? row in its §6 tracker) and it is compounding: every new endpoint PR lands in the god-module, widening the conflict surface and making the split more expensive each week. The 3-file extraction in #390 is now unmergeable.
57   - proposed fix: Close #390 and re-cut the orchestration.py extraction fresh from origin/master (small, ~1 session), or rebase it now. Then continue the split (CLI handlers -> backend/cli/, new /api/source|/api/assets|/api/capabilities routes -> backend/api/) before more endpoints accrete. Update the audit doc's stale '~1450 lines' sizing while at it.
58   - verifier notes: Minor sharpenings only: (a) git log --since=2026-06-25 -- backend/main.py actually shows 10 commits (adds #387 repo-wide ruff format ? itself a likely source of #390's conflicts ? and #380), so ">=8 PRs" is understated; (b) the mypy quarantine block is mypy.ini:15-16 ([mypy-backend.main] header at 15, ignore_errors at 16); (c) the audit doc's "1448 lines" statement is in the §Changelog 2026-06-25 "Accuracy fixes" bullet (~line 278), and the stale "~1450" sizing also appears at lines 69, 198, and 247 ? all four spots need the size refresh the fix proposes.
59   - verifier reasoning: Every load-bearing claim reproduced. (1) `gh pr view 390` returns state=OPEN, mergeable=CONFLICTING, mergeStateStatus=DIRTY, updatedAt=2026-06-26T14:37:18Z, exactly 3 files: backend/main.py (+19/-595), backend/orchestration.py (ADDED, +635), mypy.ini (+6). (2) Current working tree (branch master, HEAD e410136 = PR #413): `wc -l backend/main.py` = 2198; 1448?2198 is +51.8% ? the claimed 52%; the 1448 baseline is documented in docs/CODE_CLEANUP_AUDIT_2026-06-24.md changelog ("main.py size '~1300' ? '~1450' (now 1448 lines)", 2026-06-25 entry, ~line 278-279). (3) Not fixed: backend/orchestration.py does not exist in the tree, and git log --since=2026-06-25 -- backend/main.py shows the file only grew ? commits for PRs #413, #412, #409, #408, #407, #406, #405, #400 (the claimed 8; the log actually lists 10 incl. #387 ruff-format and #380). (4) grep counts on backend/main.py: 18 lines matching `^@app\.` and 45 `print(` occurrences ? exactly as claimed. (5) mypy.ini:15-16 = `[mypy-backend.main]` / `ignore_errors = True` (quarantine intact). (6) Audit doc §6 tracker line 198: P10 row "D1: Split main.py god-module (~1450 lines ? smaller modules)" is ?; the only other ? row is line 205 (P16+ Phase 3 bucket), so P10 is indeed the sole non-Phase-3 open item; D1 is rated HIGH at doc line 69. Severity P1 is honest for a tech-debt audit: the concrete failure is already materialized (the 635-line extraction in #390 is unmergeable today), and it compounds ? 8 endpoint-adding PRs landed in the god-module in the week since the PR was cut, all inside a mypy ignore_errors module, so each week of delay increases rework cost and unchecked-type surface. Not owner-gated: internal refactor, no serving go-live / corpus-promotion / spend / licensing implications ("P10" here is the audit tracker row, not Studio P10). Proposed fix (close-and-recut or rebase from origin/master, then continue the split per the doc's own plan at line 247, and fix the stale ~1450 sizing) is sane and correctly scoped; effort M is right given the fix includes continuing the split beyond the re-cut.
60
61   ### R1-02 [P1/M] critical-fix ? badminton-highlight-indexer
62   **/api/process re-runs the paid Gemini segmenter for an already-processed video (re-bill on refresh/'Try again') ? still unfixed at HEAD**
63   - anchor: `backend/main.py:1172` · tracked in: Khelsutra/badminton-highlight-indexer#340 (also #330) · finder: engine:correctness
64   - evidence: backend/main.py:1171-1172 computes video_id and `was_reingest = db.get_video(video_id) is not None`, but was_reingest is only used for the report/destroy-after-confirm; there is NO reuse/skip branch ? the segmenter always runs (main.py:1251-1255). Verified against gh: issue #340 OPEN ('re-runs the full (paid) segmenter anyway'), plus #330 (MD5 segmentation reuse) OPEN.
65   - why it matters: A page refresh or SPA 'Try again' on an already-indexed video (same MD5) re-bills Gemini (~$0.10/8-min match) and burns a multi-minute compute run for zero new information. Live-triggered once in the alpha (2026-06-22, per #340).
66   - proposed fix: In _execute_processing, when db.get_video(video_id) exists with status='processed' and existing play_segments (and the request doesn't explicitly force reprocess), return the stored segments as the job result instead of dispatching the segmenter; add a `force` flag to ProcessRequest for deliberate re-ingest.
67   - verifier notes: Severity P1 is honest: concrete failure scenario is a user-reachable one-click ("Try again" after the SPA's 10-min poll ceiling, live-triggered 2026-06-22 per #340) that silently re-bills a metered Gemini key ? and per #340 the M8 cost ledger does not meter Gemini yet, so the re-bill is invisible. Proposed fix is sane and correctly scoped: the skip condition must require status=="processed" AND existing play_segments (a prior "failed" record ? main.py:1196-1201 ? must still re-run), and the added `force` flag preserves the existing destroy-after-confirm re-ingest path (main.py:1233-1235, 1278). Not owner-gated: the fix reduces real cloud spend rather than incurring it, and the owner already specified the desired behavior + acceptance criteria in OPEN issue #340. Effort M is right (endpoint logic + ProcessRequest field + a no-Gemini-call regression test). One duplicate-tracking note: #330 covers the same

reuse mechanism as an optimization; a fix should close both.
68   - verifier reasoning: Verified against origin/master at HEAD e410136 (pulled, "Already up to date"). (1) Code behaves as claimed: backend/main.py:1171-1172 computes `video_id = compute_video_id(local_video)` then `was_reingest = db.get_video(video_id) is not None`; the ONLY other use of `was_reingest` in `_execute_processing` is main.py:1327, where it is passed to `emit_indexer_report` as a report field. There is no reuse/skip/short-circuit branch: after validation passes (main.py:1210-1214 upserts status "processed"), the flow unconditionally dispatches segmentation ? cloud via `_maybe_dispatch_remote` (main.py:1241-1248) or in-process `segmenter.process_video` (main.py:1251-1255). ProcessRequest (main.py:313-324) has no `force`/`reprocess` field (only video_path, player_pool, max_frames, segmenter, locale, compute). (2) Not fixed: `git log --oneline -15 -- backend/main.py` shows only unrelated commits (#413, #412, #409, #408, #407, #406, #405, #400, ...); no idempotency/reuse change. (3) Cost claim is real in served config: config.json:26 sets `"default_segmenter": "gemini"` (the paid path is the production default; the "motion" fallback at main.py:1181-1183 is only the code-level default). (4) Tracking verified via gh: issue #340 OPEN (title and body match the finding verbatim, including the 2026-06-22 live trigger and ~$0.10/8-min cost) and #330 OPEN (MD5 segmentation reuse).
69
70   ### R1-03 [P1/M] critical-fix ? badminton-highlight-indexer
71   **/api/source re-downloads the ENTIRE remote video from the bucket on every HTTP request (each browser Range request) into a fresh, never-deleted temp dir**
72   - anchor: `backend/main.py:646` · tracked in: ? · finder: engine:correctness
73   - evidence: backend/main.py:644-651: get_source calls storage.acquire_local_input(rec['filepath'], ...) on EVERY request; storage/__init__.py:89-90: for a non-local ref this does `tempfile.mkdtemp(prefix='rally-input-')` + full fetch ? a new dir + full download per call, never removed. Proxy warming is explicitly local-only (main.py:655-657), so a remote-source video never gets a proxy. FileResponse under fastapi>=0.111 (pyproject.toml:28) serves Range ? a browser scrubbing a <video> issues many separate requests to the same URL.
74   - why it matters: Direct cloud-egress cost leak plus a disk-fill denial-of-service of the appliance, reachable through a supported flow (remote-registered assets + P8 annotate-in-UI).
75   - proposed fix: Cache the fetched local copy keyed by video_id (e.g. under <output>/proxies or a per-video workspace source/ dir) and reuse it across requests; extend proxy warming to remote sources; delete or reuse the scratch dir instead of mkdtemp-per-request.
76   - verifier notes: Path correction: the storage citation is backend/storage/__init__.py:89-90 (not storage/__init__.py). Sharper failure scenario: store a 4 GB match via POST /api/assets with an s3://gs:// medium (P4, #409), open the P8 annotate UI, scrub ~10 times ? ~40 GB bucket egress (~$4-5 real cost) and ~40 GB of never-deleted rally-input-* temp dirs per session; each seek also stalls behind a full multi-GB download, so the feature is effectively unusable for remote sources. Additional color for the fix: the engine free-space guard (require_free_bytes, #407) is not applied to this download path, and the cached copy should live under a guarded root; the finding's proposed fix (cache keyed by video_id, reuse across requests, extend proxy warming to remote) is sane and correctly scoped.
77   - verifier reasoning: backend/main.py:646 calls storage.acquire_local_input(rec["filepath"], ...) on every GET /api/source/{video_id} when no proxy exists (proxy check main.py:641-642). backend/storage/__init__.py:89-90: non-local refs get a fresh tempfile.mkdtemp(prefix="rally-input-") + full fetch per call; docstring (lines 80-82) admits the copy "relies on OS temp reclamation" (nonexistent on Windows %TEMP%), and a repo-wide grep for "rally-input" shows no cleanup path. Remote filepaths are a supported state: POST /api/assets registers the cloud stored_uri as filepath (main.py:838, 855; comment at main.py:1680 lists gs://, s3://, gdrive://). Proxy warming is explicitly local-only (main.py:655-657, docstring 628-629 "remote-source proxying is a follow-up"), so remote sources never escape the full-fetch path. Installed fastapi 0.136.1 / starlette 1.0.0 FileResponse honors Range, so a scrubbing <video> issues many independent requests, each a full-object download. git log -15 -- backend/main.py (through #413) shows no fix after #406 introduced this. Not owner-gated: the fix is plain engineering that reduces cloud spend.
78
79   ### R1-04 [P1/M] tech-debt ? badminton-highlight-indexer
80   **Import cycles: extracted API modules reach back into backend.main globals at call time; config layer imports pipeline layer**
81   - anchor: `backend/api/job_worker.py:75` · tracked in: docs/CODE_CLEANUP_AUDIT_2026-06-24.md P5 note + D1/P10 follow-up (line 221) · finder: engine:structure
82   - evidence: AST scan of backend/ found 5 strongly-connected components. (1) backend.main

<-> backend.api.job_worker <-> backend.api.uploads: job_worker.py:75,98,214,234,251 and uploads.py:47 do call-time 'import backend.main as _main' to read main.py's module globals db_instance/global_config (main.py:279-280); main.py:292 re-exports job_worker names back onto itself. Documented as a transitional shim (CODE_CLEANUP_AUDIT P5 note, line 221) but it makes main.py load-bearing for every extracted module and keeps backend.main AND backend.worker.__main__ mypy-quarantined (mypy.ini:15-23). (2) Layering inversion: backend/config/models.py:249 imports segmenter_registry from backend.pipeline.segmenters.base inside a validator, while base.py:15 (_normalize_config) imports AppConfig back from config.models ? config validation now depends on pipeline registration side effects. (3-5) eval-package cycles (golden_fixtures<->golden_real_fixtures, ablation<->ablation_pdf, annotations<->gt_loader) are lazy and explicitly documented (golden_fixtures.py:1005-1028) ? lower priority.
83   - why it matters:
84   - proposed fix: Kill the shim as the concrete step-2 of D1: move db_instance/global_config onto app.state (create_app already exists per P5) and pass config/db explicitly to job_worker/uploads, then lift backend.main and backend.worker.__main__ out of mypy quarantine. For the layering inversion, move the segmenter-name validation out of the pydantic validator into a startup check (backend/config/startup.py) so config/ no longer imports pipeline/.
85   - verifier notes: Mechanism correction to the evidence: job_worker's five call-time imports do NOT read db_instance/global_config ? job_worker already takes config/db as explicit params (job_worker.py:89,205,230,247; mypy.ini:14 notes it was lifted from quarantine) and resolves FUNCTION seams (ProcessRequest, _execute_compile, _execute_processing, JobWaiting, _arm_wake_timer, _get_executor, _MAX_DRAIN_WAKE_SEC) through the backend.main module object per the documented test-monkeypatch contract (job_worker.py:8-16, main.py:286-291). Only uploads.py:49 reads a main global (global_config). So the app.state migration for db/config is mostly about uploads + main.py itself; the harder remaining work is redesigning the monkeypatch seam contract and its tests ? effort is top-end M. Sharper failure scenario: any alternative entrypoint (new console_script, uvicorn factory) that builds the app without running main() leaves backend.main.global_config=None and uploads silently falls back to default upload dir/size limits (uploads.py:49-52), ignoring operator security config ? same class as the already-documented python -m backend.main incident (main.py:2188-2195) that hung every queued job. Layering inversion is slightly worse than claimed: the validator's lazy import (models.py:249) transitively executes segmenters/__init__.py:1-7, importing ALL concrete segmenters, so constructing AppConfig anywhere drags the full pipeline in; and SegmenterRegistry.register (base.py:66-70) re-enters AppConfig construction during registration with errors swallowed by base.py:19's bare except.
86   - verifier reasoning: Verified in current tree (HEAD e410136, pulled clean). Cycle 1: backend/main.py:292-302 imports backend.api.job_worker at module load (re-exports); job_worker.py:75,98,214,234,251 do call-time 'import backend.main as _main' (grep-confirmed all 5); uploads.py:47-49 call-time imports and reads _main.global_config; globals defined main.py:279-280, set at main.py:101-103 and 1981-1982; mypy.ini:15-16 quarantines backend.main, :22-23 backend.worker.__main__. Cycle 2: config/models.py:244-256 validator imports segmenter_registry from pipeline/segmenters/base; base.py:14-18 imports AppConfig back in _normalize_config; importing base initializes segmenters/__init__.py:1-7 which imports every concrete segmenter (motion.py:243 etc. register as import side effects), so AppConfig construction depends on pipeline import health ? and register() (base.py:66-70) re-enters AppConfig via segmenter_cls(None, {}) with failures swallowed by base.py:19 bare except. Eval cycles lazy+documented (golden_fixtures.py:1008-1013). Not already fixed: git log for the four files shows only feature commits (#405-#413). Severity honest: the failure class already bit ? main.py:2188-2195 documents the python -m backend.main dual-module incident (backend.main.db_instance=None, drain crashed silently, jobs hung 'queued' forever), patched only for that entry path (main.py:2196-2198); uploads.py:49-52 getattr-defaults mean any entrypoint that skips main() silently runs uploads with default dir/limits. Tracked_in accurate: audit doc line 221 is verbatim the P5 shim note. Not owner-gated: only owner question is D1 phase sequencing, none of the listed gates. Fix sane: create_app exists (main.py:2184), handlers already use app.state (mypy.ini:12-13), backend/config/startup.py exists as the startup-check home.
87
88   ### R1-05 [P1/M] tech-debt ? badminton-highlight-indexer
89   **Path-jail rejects gs://,s3://,gdrive-api:// before scheme resolution ? bucket ingest impossible on the exposed alpha box (issue #327 still open)**
90   - anchor: `backend/main.py:1376` · tracked in: https://github.com/Khelsutra/badminton-highlight-indexer/issues/327 · finder: engine:security-config
91   - evidence: main.py:1376 calls `validate_input_path(req.video_path,

allowed_root=allowed_root)` BEFORE `storage.resolve_input_ref` at main.py:1379; backend/utils/paths.py:45-55 `resolve_under_root` realpaths the raw URI so `gs://bucket/x.mp4` escapes the jail -> 400. Startup check startup.py:106-119 makes the jail mandatory when exposed (EXPOSED_NO_VIDEO_ROOT), so an exposed engine can never ingest from a bucket. gh: issue #327 OPEN (verified 2026-07-02), proposing scheme-aware validation.
92   - why it matters: Blocks the alpha CUJ (large-video ingest via gs:// instead of 500 MB browser uploads) and is a stale-open thread since before 06-26; the fix must be done carefully to avoid reopening the arbitrary-local-file hole the jail exists for.
93   - proposed fix: Implement #327 option (b): jail only after resolve_input_ref, applying resolve_under_root exclusively to backend=='local' refs; keep traversal/symlink/absolute rejection for local paths and add both allow (bucket URI accepted) and reject (local traversal still 400) tests, updating tests/test_api_endpoints.py::test_process_hosted_mode_rejects_nonlocal_uri.
94   - verifier notes: Minor corrections: (1) startup check path is backend/config/startup.py:106-119, not startup.py (lines exact). (2) The "500 MB browser uploads" figure in why_it_matters comes from issue #327's body; the repo's actual browser-upload cap is security.max_upload_mb default 4096 MB (backend/api/uploads.py:51, backend/config/models.py:339) ? 500 MB is the Gemini provider per-clip cap (backend/providers/gemini.py:23). Immaterial to the finding (multi-hour matches exceed 4 GB anyway; docs/VIDEO_LOCALITY_MODEL.md:26 "The 10 GB problem is real"). (3) Fix scope additions: there is a second resolve call-site (CLI --video-path, see tests/test_api_endpoints.py:922) ? verify whether it needs the same scheme-aware treatment; ensure unknown schemes still ValueError?400 via resolve_input_ref as the sole scheme gate. (4) Caution, not a gate: the change relaxes a deliberately pinned security contract on the exposed box; #327 records the owner's explicit want, but the option (a) vs (b) choice and the PR should get owner sign-off before landing since it alters exposed-serving posture.
95   - verifier reasoning: Personally reproduced from current code. (1) Ordering: backend/main.py:1376 calls validate_input_path(req.video_path, allowed_root=allowed_root) BEFORE storage.resolve_input_ref at main.py:1379-1381; ValueError maps to HTTP 400 at main.py:1382-1383. (2) Mechanism: backend/utils/paths.py:58-68 routes any request with allowed_root set through resolve_under_root, which os.path.realpath()s the RAW string (paths.py:47) ? 'gs://bucket/x.mp4' realpaths to a cwd-relative path, commonpath != root (paths.py:50-54) ? ValueError ? 400. The code itself documents this at main.py:1387-1389: "a configured allowed_video_root (hosted mode) rejects a non-local URI as out-of-root". (3) Pinned by test: tests/test_api_endpoints.py:903-912 asserts gs://, s3://, gdrive:// all ? 400 when allowed_video_root is set ("Pins the security contract"). (4) Exposure mandate: backend/config/startup.py:106-119 emits LEVEL_ERROR EXPOSED_NO_VIDEO_ROOT when edge_auth_enabled and allowed_video_root unset ? so an exposed engine must have the jail, making bucket ingest impossible there. (5) Not fixed: git log -15 -- backend/main.py (head e410136, #413) shows no commit addressing this; behavior present in current tree. (6) gh issue view 327: state OPEN, created 2026-06-22, title/body match the finding exactly, including the owner's explicit want ("ingest large videos via gs:// on the hosted box") and options (a)/(b). Concrete failure: exposed alpha box (edge_auth_enabled=true, allowed_video_root=/jail per boot requirement) receives POST /api/process {"video_path":"gs://bucket/match.mp4"} ? 400 "path ... escapes the allowed root"; only ingest path left is the capped browser upload (backend/api/uploads.py:51, security.max_upload_mb) ? blocks the tracked alpha large-video CUJ. Severity P1 honest (owner-wanted CUJ blocked, open since 2026-06-22, workaround capped/painful per docs/VIDEO_LOCALITY_MODEL.md:19-26). Proposed fix (option b: jail only backend=='local' after resolve_input_ref, keep null-byte check universal and traversal/symlink jail for local, add allow+reject tests, update the pinned test) matches the issue's own proposal and is correctly scoped; effort M is right.
96
97   ### R1-06 [P1/S] doc-rot ? khelsutra (avidullu/khelsutra)
98   **Docs claim the retired droplet alpha is LIVE (droplet deleted 2026-06-25)**
99   - anchor: `NEXT_STEPS.md:12` · tracked in: partially in issue #97 (item 2: update DEPLOY_ALPHA.md); the NEXT_STEPS.md droplet claim is untracked · finder: khelsutra:issues-triage
100   - evidence: NEXT_STEPS.md:12-13 ('Where we are (updated 2026-07-01)') still says the site is 'live + browsable on the DO droplet -> http://168.144.126.176:8787'; deploy/DEPLOY_ALPHA.md:3 says 'Status: LIVE (deploy live-verified on the droplet 2026-06-22)' and :29 'This runbook ships the split topology'; docs/LOCAL_APPLIANCE_ARCHITECTURE.md:176 'The alpha stand-up is now LIVE (2026-06-22)'. Project memory (MEMORY.md:63, session-handoff.md:19): droplet claude-do-rally-test/168.144.126.176 RETIRED+deleted 2026-06-25; alpha paused pending Cloud Run. nslookup shows api./app.khelsutra.guru still resolve via Cloudflare to a tunnel with no origin (pending CNAME/Access cleanup).
101   - why it matters:

- proposed fix: One doc PR: mark DEPLOY_ALPHA.md status RETIRED/PAUSED with a pointer to the Cloud Run plan, fix NEXT_STEPS.md 'Where we are' (droplet gone, site is CF-Workers-only), and correct LOCAL_APPLIANCE_ARCHITECTURE.md:176. Also note the single-origin repoint that superseded the split topology (issue #97). The 2026-07-01 refresh (#139) touched docs but kept the stale claim, so this will keep misleading ramp-up reads.
- verifier notes: Minor anchor correction: the 'Where we are (updated 2026-07-01)' header is NEXT_STEPS.md:5; the droplet-live claim spans lines 12-13 (the finding's line 12 anchor is correct for the URL text). Severity P1 stands: NEXT_STEPS.md's 2026-07-01 stamp is NEWER than the handoff's 2026-06-25 correction, so ramp-up readers reconciling the two will trust the wrong doc; worse, DEPLOY_ALPHA.md §1 (line 35) directs deploying the engine + secrets onto 168.144.126.176 ? an IP the owner no longer controls and DO can recycle. Fix is sane, doc-only, and not owner-gated (it records already-made owner decisions: retirement 2026-06-25 + single-origin repoint per issue #97); one caution ? the fix should point to the Cloud Run move as 'paused pending' rather than declare a new serving decision, which would be owner-gated. Did not independently re-verify the finder's nslookup/DNS claim; the finding stands without it.
- verifier reasoning: Working tree at HEAD f9eea14: NEXT_STEPS.md:5 'Where we are (updated 2026-07-01)' + :12-13 'live + browsable on the DO droplet -> http://168.144.126.176:8787 (systemd khelsutra-site)'; deploy/DEPLOY_ALPHA.md:3 'Status: LIVE (deploy live-verified on the droplet 2026-06-22...)' + :29 'This runbook ships the split topology'; docs/LOCAL_APPLIANCE_ARCHITECTURE.md:176 'The alpha stand-up is now LIVE (2026-06-22): the engine booted on the droplet (168.144.126.176)'. Ground truth: memory/khelsutra-site-droplet-parity.md:3,10-14 (droplet DELETED 2026-06-25, data archived, doctl delete done, monitor removed via PR #117), session-handoff.md:19 ('the prior LIVE alpha on the droplet is GONE ? alpha paused pending the Cloud-Run move'), MEMORY.md:63. Not fixed: git log shows the last commit touching all three files is 9223ba9 (#139, the 2026-07-01 refresh) and the stale text persists at HEAD. Tracking verified: gh issue 97 is OPEN, item 2 covers only DEPLOY_ALPHA.md (single-origin update); the NEXT_STEPS.md droplet claim is untracked. Issue #97 also confirms split topology was superseded by the 2026-06-22 single-origin repoint, so DEPLOY_ALPHA.md:29 is doubly stale.

R1-07 [P1/M] critical-fix ? khelsutra (avidullu/khelsutra)

Issue #111 still fully valid: refresh loses the in-flight job; progress static for minutes

- anchor: `web/src/hooks/useIngestJob.ts:117` · tracked in: khelsutra#111 · finder: khelsutra:issues-triage
- evidence: useIngestJob.ts:117-128 ? job_id lives only in the poll closure (no localStorage/URL persistence); App.tsx:71-73 seeds state only from ?video= (set on done, App.tsx:153). A refresh mid-job lands the user back on the ingest form while the server job keeps running. Progress is the coarse free-text job.progress (useIngestJob.ts:106); no heartbeat/animation beyond a spinner chip (IngestPanel.tsx:337-358).
- why it matters:
- proposed fix: SPA half is Claude-doable now: persist the active job_id (?job=<id> + localStorage) and on load resume polling to the existing ?video= handoff; add elapsed-time/heartbeat copy. Engine half (granular 'segmenting chunk 3/7' from backend/pipeline/segmenters/gemini.py via the job-progress hook) is a small cross-repo change. Per the issue this is 'the most likely way an early tester loses a working job'.
- verifier notes: Failure scenario, sharpened from issue #111's field report: an ~8-min upload spends ~5 min in Gemini segmentation with near-static "processing" text; the tester refreshes (natural when nothing moves), lands on the ingest form with no trace of the running job, and either re-submits ? a duplicate job, potentially on paid cloud GPU via the compute picker ? or abandons believing it failed; the finished video only appears later in the library with no pointer back. Fix note: the backend is already refresh-safe (jobs persisted in SQLite), so the SPA half (persist ?job=/localStorage, resume polling into the existing ?video= handoff, heartbeat/elapsed-time copy) is standalone and Claude-doable now; only the granular-progress half touches the sibling badminton-highlight-indexer repo.
- verifier reasoning: useIngestJob.ts:117-128 ? job_id is a closure-local (`let jobId` at :117, used only by poll(:128)); grep of web/src confirms no localStorage/URL persistence of any job id (only annotate flag, wizard key, i18n lang). App.tsx:71-73 seeds videoId solely from ?video=, written only on job-done via loadVideo (App.tsx:150-154 ? useIngestJob.ts:96-98), so a mid-job refresh renders the ingest form (App.tsx:338-343) while the server job keeps running. Progress is coarse free-text job.progress (useIngestJob.ts:106) shown as a spinner chip (IngestPanel.tsx:337-348), no heartbeat/elapsed time. git log -15 on both files (latest #140, #138, #128?) shows no fix; issue khelsutra#111 is OPEN and documents the identical gaps from a live 2026-06-22 alpha run. Engine path backend/pipeline/segmenters/gemini.py exists in the sibling badminton-highlight-indexer repo as the finder scoped it. Not owner-gated: pure client

resiliency, no go-live/corpus/licensing/spend/legal decision.
114
115   ### R1-08 [P1/S] tech-debt ? khelsutra (avidullu/khelsutra) ? SPA web/ + marketing site/
116   **Engine's vendored SPA bundle is 8 feature-PRs stale ? shipped appliance UI lacks the entire storage/annotation wave**
117   - anchor: `badminton-highlight-indexer/backend/ui/ui_version.json:3` · tracked in: ? · finder: khelsutra:code
118   - evidence: backend/ui/ui_version.json pins spa_commit ed33a515 built_at 2026-06-22T17:59:59Z, but khelsutra web/src has 8 newer PRs all merged 2026-07-01 (git log: f9eea14 #140 fps-in-library, fb2f561 #138 FFmpeg banner, 8667dbf #137 medium picker, cc06c32 #128 AnnotatePanel, 9e9df72 #127 persistent name, 926b53a #126 vendored annotator, 39c1349 #124 library, d0285cd #122 e2e harness). grep for 'video-library'/'library.title' in backend/ui/assets/index-ukuS9wxa.js = 0 hits. Meanwhile the engine ALREADY serves the matching endpoints: /api/assets (backend/main.py:727), /api/source (main.py:623), /api/annotations (main.py:663), and PRs #411-413 (e410136, 81a78bf) enriched /api/videos specifically for the new SPA library ? so a wheel install serves an API whose UI can't reach its own features. The API_VERSION skew guard doesn't fire (still 1, main.py:408); nothing detects content staleness.
119   - why it matters:
120   - proposed fix: Re-run badminton-highlight-indexer/scripts/bundle_ui.sh against khelsutra@f9eea14 and commit the refreshed backend/ui/ (the script's own convention: 'Refresh the snapshot by re-running this on an SPA release; commit the diff'). Then add a cheap staleness signal: a CI/doctor check comparing ui_version.json spa_commit against khelsutra origin/master (warn when >N commits behind), so the bundle-sync convention can't silently rot again.
121   - verifier notes: Minor corrections only: API_VERSION is at backend/main.py:407, not 408; and web/ actually has 10 commits since the pin (the 8 cited feature/test PRs plus docs #139/#123). Sharper failure scenario: a user who pip-installs the wheel and runs `indexer --app` gets an appliance UI with no video library, no annotate panel, no storage-medium picker, and no FFmpeg-missing banner, while the very same wheel serves /api/assets, /api/source, /api/annotations, and the just-enriched /api/videos (fps/resolution/match_name from #412/#413) ? the engine's two newest commits are dead payload for every wheel user. P1 stands because both repos' latest work (engine #411-413, SPA P5-P8) targets features unreachable in the only shipped artifact. Fix scope is right; note the CI staleness check needs network access to the khelsutra remote, so a doctor-time warning may be the cheaper variant.
122   - verifier reasoning: backend/ui/ui_version.json:3 pins spa_commit ed33a515 (built_at 2026-06-22T17:59:59Z, line 6); khelsutra `git log ed33a515..HEAD -- web/` shows all 8 cited PRs (#140 f9eea14, #138 fb2f561, #137 8667dbf, #128 cc06c32, #127 9e9df72, #126 926b53a, #124 39c1349, #122 d0285cd) merged 2026-07-01. Vendored bundle backend/ui/assets/index-ukuS9wxa.js has 0 occurrences of 'video-library' (the 2 'Your videos' hits are privacy-policy prose), while current web/src/components/VideoLibrary.tsx:44,48 uses t("library.title") and data-testid="video-library". Engine endpoints exist as claimed: /api/source backend/main.py:623, /api/annotations :663, /api/assets :727, and engine HEAD e410136/#413 + 81a78bf/#412 enriched /api/videos for the new library. API_VERSION = 1 at backend/main.py:407 with a comment that additive endpoints need no bump, so the skew guard cannot fire; no spa_commit staleness check exists in backend/. Not fixed: last commit touching backend/ui/ is 8bcf6ce (2026-06-22). scripts/bundle_ui.sh header confirms the refresh-and-commit convention the fix proposes. Not owner-gated: documented routine snapshot refresh; AnnotatePanel is flag-gated in the SPA (#128) so bundling is safe.
123
124   ### R1-09 [P1/S] doc-rot ? khelsutra (avidullu/khelsutra) ? SPA web/ + marketing site/
125   **DEPLOY_ALPHA.md claims 'Status: LIVE' on a droplet retired 2026-06-25; GPU_VM_SERVING.md sources its secrets from that droplet**
126   - anchor: `khelsutra/deploy/DEPLOY_ALPHA.md:3` · tracked in: khelsutra #97, #114 (topology); droplet retirement itself untracked in-repo · finder: khelsutra:code
127   - evidence: deploy/DEPLOY_ALPHA.md:3 '**Status:** `LIVE` (deploy live-verified on the droplet 2026-06-22...)'; :35 names 'the CPU droplet 168.144.126.176' as the engine box. The droplet was retired 2026-06-25 (khelsutra commit 5ffb750 #117: 'retire the live-parity monitor ? droplet mirror decommissioned'); grep -i 'retired|decommission' over the repo's docs = 0 hits, so no doc records it. deploy/GPU_VM_SERVING.md §0 tells the operator to copy the CF-Access AUD from 'the droplet's /srv/khelsutra/config.json' and the tunnel token from 'systemctl cat cloudflared' on the droplet, and calls the droplet a 'hot fallback' ? all impossible now. Secondary rot: DEPLOY_ALPHA.md:143 claims the SPA 'shows a friendly engine offline banner (/api/health ping)' but web/src contains zero references to /api/health (grep -i health = no matches); and open issue #114 (Access sign-in broken) + #97 (split-origin breaks CF Access) contradict the runbook's shipped topology.
128   - why it matters:

- proposed fix: One doc PR: flip DEPLOY_ALPHA.md status to RETIRED/HISTORICAL with a pointer to the retirement date and to issues #97/#114; rewrite GPU_VM_SERVING.md prerequisites to source the Gemini key/AUD/tunnel token from their canonical stores (Drive / CF dashboard) instead of the dead droplet; delete or soften the unimplemented /api/health-banner claim. The related api.khelsutra.guru CNAME + CF-Access application cleanup is Cloudflare-dashboard work (owner-gated) ? note it in the doc as pending.
- verifier notes: Two refinements. (a) "all impossible now" is slightly overstated: per 5ffb750's message the droplet's data was archived to gs://khelsutra-rally-corpus/archive/droplet-claude-do-rally-test-20260625/, so /srv/khelsutra/config.json (and its cf_access_aud) is likely recoverable from that archive, and the tunnel token is recoverable from the Cloudflare dashboard since the tunnel is token-managed (GPU_VM_SERVING.md:24-26). The doc rewrite should point at the gs:// archive + CF dashboard as the canonical sources ? strictly better than the finder's "Drive / CF dashboard" suggestion for the AUD. (b) DEPLOY_ALPHA.md:143 is partial rot, not total: the SPA does degrade gracefully when the engine is unreachable (SetupWizard offline branch, web/src/components/SetupWizard.tsx:84-142), but via a capabilities fetch, and there is no persistent "engine offline" banner and zero /api/health references ? so soften rather than delete the claim. Also worth citing commit 5ffb750 / PR #117 and the "Cloudflare is now the single canonical host" line in the doc PR as the retirement record, since no .md currently captures it.
- verifier reasoning: Personally reproduced every element of the claim in the current working tree (khelsutra @ f9eea14). (1) deploy/DEPLOY_ALPHA.md:3 reads "**Status:** `LIVE` (deploy live-verified on the droplet 2026-06-22...)" and :35 names "the CPU droplet `168.144.126.176`" as the engine box; :143 claims the SPA "shows a friendly 'engine offline' banner (`/api/health` ping)". (2) Commit 5ffb750 (2026-06-25, PR #117) states verbatim: "That droplet has been decommissioned (its data archived to gs://khelsutra-rally-corpus/archive/droplet-claude-do-rally-test-20260625/)... Cloudflare is now the single canonical host" ? it only removed .github/workflows/parity.yml + site/scripts/parity-check.sh, touching neither deploy doc. (3) grep -i 'retired|decommission' over all *.md in the repo = 0 matches, so no doc records the retirement. (4) deploy/GPU_VM_SERVING.md:24 still says "the droplet stays as a hot fallback"; :37-40 instruct sourcing the CF-Access AUD from "the droplet's /srv/khelsutra/config.json" and the tunnel token from "systemctl cat cloudflared" on the droplet ? and that file was last edited 2026-06-29 (ca8bfcc #119), FOUR DAYS AFTER the decommission, without fixing this, so NOT already-fixed. (5) grep over web/src: zero references to /api/health anywhere (only FfmpegBanner.tsx exists as a banner; SetupWizard.tsx:84-142 has an offline message driven by a capabilities fetch, not a /api/health ping), confirming the :143 rot. (6) gh: issues #114 (allowlisted email couldn't sign in, real demo 2026-06-23) and #97 (split-origin breaks CF Access ? single-origin needed) are both OPEN, contradicting "Status: LIVE ... only a rotated Gemini key + a browser OTP login remain". Concrete failure: this is a docs-as-memory, multi-session project ? the next serving session following GPU_VM_SERVING.md §0 is blocked at prerequisites trying to SSH a nonexistent box for secrets, and DEPLOY_ALPHA.md's LIVE status + "hot fallback" cutover-safety claim would drive wrong go-live/rollback decisions for the actual alpha. P1 is honest for the primary serving runbook; effort S (one doc PR) is right. The doc PR itself is not owner-gated; only the Cloudflare-dashboard CNAME/Access cleanup is, and the proposed fix already scopes that as a note.

R1-10 [P1/S] hygiene ? sports-data-collector

Stray nested clone sports-data-collector/sports-data-collector/ pollutes tooling; safe to delete

- anchor: `sports-data-collector/sports-data-collector/:` · tracked in: ? · finder: sdc:audit
- evidence: Untracked dir is a FULL second clone of the same repo: remote = https://github.com/avidullu/sports-data-collector.git (pre-org-move URL; parent origin is https://github.com/Khelsutra/sports-data-collector.git). HEAD = 031e410 'feat(annotation): richer Roora schema (#29)' (~Jun 19; working-tree files mtime Jun 19 21:38, .git mtime Jun 29 14:41 from a later fetch that moved its origin/master to a152cf5). Verified zero unique work: `git merge-base --is-ancestor 031e410 HEAD` in the parent succeeds, `git status --short` clean, `git stash list` empty, only branch is master. Size ~1.7 MB (811 KB .git). It contains its own tests/, src/, .github/, and data/sports_metadata.db (tracked at that commit, unmodified). Tooling pollution measured: `ruff check .` from repo root reports 39 errors vs 24 when scoped to src/tests/cleanup.py ? the extra 15 come from the nested clone; bare `pytest` rootdir collection and setuptools package discovery would also pick it up.
- why it matters:
- proposed fix: Delete the entire nested directory (rm -rf sports-data-collector/sports-data-collector/). Nothing in it is unique: HEAD is an ancestor of current master, tree is clean, no stashes/branches. Root cause is almost certainly a `git clone` run with cwd already inside

the repo (~Jun 19); worth adding `sports-data-collector/` to .git/info/exclude only if deletion is deferred. READ-ONLY audit ? not deleted; owner or a follow-up session should remove it.
139   - verifier notes: Sharper failure scenario than the finder's: beyond the +15 phantom ruff errors, both tests/ dirs share module basenames with no __init__.py, so bare `pytest` from repo root fails at collection with import-file-mismatch errors ? i.e., the owner's pre-LGTM gate ("full test suite green", "ruff check . clean") is actively corrupted for any local session until the dir is deleted; agents using root-wide Grep/Glob also risk editing the stale nested copy. Two cautions for the fixer: (a) parent working tree currently has an unrelated modified .gitignore (adds `artifacts/` ? likely sibling/owner WIP per global instructions; do not clobber or commit it while deleting the nested dir); (b) nested .git mtime is Jul 2 08:18 (today) ? something (likely the finder's fetch or background maintenance) touched it, but no new local work exists (origin/master..master empty), so deletion remains safe.
140   - verifier reasoning: Every material claim reproduced against the current tree. (1) Nested clone exists: C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector/sports-data-collector/ is a full checkout (.git, src/, tests/, .github/, data/, cleanup.py; dir mtime Jun 19 21:38) and parent `git status --short` shows it untracked (`?? sports-data-collector/`). (2) Remotes match the claim: nested `git remote -v` = https://github.com/avidullu/sports-data-collector.git (pre-org-move), parent origin = https://github.com/Khelsutra/sports-data-collector.git. (3) Zero unique work, personally re-verified: nested HEAD = 031e410 "feat(annotation): richer Roora schema (#29)"; `git merge-base --is-ancestor 031e410 HEAD` in the parent (HEAD adcd206) succeeds; nested `git status --short` clean, `git stash list` empty, only branch master, and `git log origin/master..master` empty (no unpushed commits). Size matches: du = 1.7M total, 811K .git. (4) Tooling pollution reproduced exactly: `ruff check . --no-cache` from repo root = "Found 39 errors" vs `ruff check src tests cleanup.py --no-cache` = "Found 24 errors" ? the 15-error delta is the nested clone. No guard exists: pyproject.toml (lines 1-41, entire file) has no [tool.ruff] or [tool.pytest.ini_options] section, and no pytest.ini/setup.cfg/tox.ini/ruff.toml anywhere; nested dir absent from .gitignore and .git/info/exclude. (5) The pytest claim is structurally worse than stated: tests/ and sports-data-collector/tests/ contain identically-named modules (test_batch_preflight.py, test_cli.py, test_config.py, test_cvat_parser.py, ...) and NEITHER has __init__.py, so a bare `pytest` from root would hit "import file mismatch" collection errors ? breaking the owner's mandated full-suite pre-LGTM gate locally. Not owner-gated: pure local hygiene, no serving/training/licensing/spend/ToS surface. Fix (rm -rf the nested dir) is safe and correctly scoped given the ancestor/clean/no-stash/no-unpushed proof.
141
142   ### R1-11 [P2/M] ci-infra ? badminton-highlight-indexer
143   **ci.yml/nightly.yml lane drift predicted by #384 has now materialized (link-check missing from nightly, timeout divergence, duplicated coverage floor)**
144   - anchor: `.github/workflows/nightly.yml:12` · tracked in: https://github.com/Khelsutra/badminton-highlight-indexer/issues/384 · finder: engine:ci-tests
145   - evidence: ci.yml:53-63 added a link-check job (commit af1ccbe, PR #402, 2026-06-30) that nightly.yml does not mirror; nightly.yml:12-16 still claims 'every OFFLINE guard job from ci.yml is mirrored here' and its notify-on-failure needs list (nightly.yml:234) has no link-check; timeouts have also diverged (license-scan: ci.yml:187=15min vs nightly.yml:171=10min; build-artifact: ci.yml:214=30min vs nightly.yml:197=20min); coverage floor 70 is a duplicated literal (ci.yml:145, nightly.yml:139) that nightly.yml:22-23 itself flags as not single-sourced
146   - why it matters: Issue #384 predicted exactly this: copied lanes silently drift. The drift is no longer hypothetical ? the nightly's parity claim is false, and the next coverage-floor ratchet in ci.yml would leave nightly enforcing a stale floor with nobody noticing (the nightly's whole purpose is catching rot during quiet periods).
147   - proposed fix: Do the #384 refactor now: extract the shared offline guard lanes into .github/workflows/_offline-guards.yml exposed via workflow_call with the coverage floor as an input, and have both ci.yml and nightly.yml call it (add link-check to the shared set or document why it is per-PR-only); fix the stale parity comment in nightly.yml:12-16 either way.
148   - verifier notes: Two corrections. (a) The timeout "divergence" is congenital, not materialized drift: `git merge-base --is-ancestor` shows 3a34c42 (#382, ci.yml timeouts 15/30) landed BEFORE the nightly was created (9972057, #381), and `git show 9972057:nightly.yml` already carried 10/20 ? the values never matched, so only the link-check omission + stale parity comment are new drift from af1ccbe. (b) Severity downgraded P1?P2: the sharpest real failure scenario today is weak ? tools/check_md_links validates internal repo links, which only change via commits, and every push/PR runs ci.yml's link-check on identical content, so the nightly missing that lane forfeits ~no detection ("rot during quiet periods" cannot affect a content-only check). The coverage-floor stale-ratchet failure is future-conditional (requires a ratchet PR that forgets nightly.yml:139), and the nightly's tighter license-scan timeout (10 vs 15) would at worst cause a visible spurious failure (notify-on-failure opens a tracking issue ?

noisy, not silent). What IS immediate is a false parity comment at nightly.yml:12-16. Proposed fix is sane and matches open issue #384; minimum viable slice if the M-sized refactor is deferred: add link-check to nightly + fix the comment + align timeouts (S).
149   - verifier reasoning: Verified in current tree (master, up to date with origin). (1) ci.yml:53-63 defines the link-check job (runs `python -m tools.check_md_links`, offline), added by commit af1ccbe (PR #402, 2026-06-30) per `git log -- .github/workflows/ci.yml`; `git show --stat af1ccbe` confirms it touched ci.yml but not nightly.yml. (2) nightly.yml:12-16 still asserts "every OFFLINE guard job from ci.yml is mirrored here" listing only 8 lanes (no link-check) ? the parity claim is now false; nightly.yml:234 needs list `[lint, leak-scan, typecheck, import-smoke, full-suite, golden-fixtures, license-scan, build-artifact]` also lacks link-check. (3) Coverage floor is a duplicated literal: ci.yml:145 and nightly.yml:139 both `--cov-fail-under=70`; nightly.yml:22-23 self-flags "still copied literals, not single-sourced". (4) Timeouts differ as cited: license-scan ci.yml:187=15 vs nightly.yml:171=10; build-artifact ci.yml:214=30 vs nightly.yml:197=20. (5) Not fixed: nightly.yml last commit is ffb1f53 (#391); no _offline-guards.yml or any workflow_call reusable exists (.github/workflows/ contains only ci.yml and nightly.yml). (6) `gh issue view 384` = OPEN, proposing exactly the workflow_call extraction; its own triage header says "LOW ? drift". Not owner-gated: pure CI plumbing, floor stays at 70 (respects the ratchet-only note at ci.yml:141-143), no serving/corpus/licensing/spend decisions.
150
151   ### R1-12 [P2/S] security ? badminton-highlight-indexer
152   **Default CORS allow_origins=['*'] lets any website script the localhost appliance (paid Gemini runs + local data read)**
153   - anchor: `backend/main.py:116` · tracked in: ? · finder: engine:security-config
154   - evidence: backend/main.py:114-116 `_cors_origins_from_env` returns `origins or ["*"]`; main.py:123-129 adds CORSMiddleware with allow_methods=['*'], allow_headers=['*']. The TrustedHost guard (main.py:132-156) only blocks DNS-rebinding ? a direct fetch to http://127.0.0.1:<port> from a hostile page sends Host: 127.0.0.1 (allowed) and the wildcard CORS answers its preflight, so evil.com JS can POST /api/process (no auth by default: backend/api/auth.py:126-127 returns None when edge_auth off) and trigger a paid Gemini run (main.py:1817 reads GEMINI_API_KEY; issue #340 documents re-billing) and read /api/videos responses. The UI is served same-origin (main.py:240,255 StaticFiles mounts), so wildcard CORS is not needed for the primary UX. Mitigated in current Chrome by Local Network Access prompts, NOT in Firefox.
155   - why it matters: Cost-leak plus local-data exposure reachable in normal use: a user browsing any hostile page while the appliance runs can have their Gemini key billed and their video index read. It silently defeats the P0c DNS-rebinding hardening the repo already invested in.
156   - proposed fix: Change the default to same-origin/loopback (e.g. `[]` plus skipping the middleware, or an explicit localhost dev-origin list) and keep RALLY_CORS_ALLOW_ORIGINS as the widening knob; add a startup exposure/appliance warning when the wildcard is active. Add a regression test that an evil-origin preflight is rejected by default.
157   - verifier notes: Downgrade P1?P2: the failure scenario is real but the finding itself honestly notes Chromium Private Network Access mitigates it in Chrome/Edge (dominant browser share); residual exposure is mainly Firefox against a local single-user appliance. Still a legitimate secure-default regression worth fixing ? it silently undercuts the P0c hardening and is inconsistent with the repo's own loopback-by-default posture for bind host and allowed hosts. Fix and effort (S) as proposed are accurate.
158   - verifier reasoning: All cited code exists and behaves as claimed in current HEAD (e410136). backend/main.py:109-116 `_cors_origins_from_env` returns `origins or ["*"]`; main.py:123-129 adds CORSMiddleware with allow_origins from that fn, allow_methods=['*'], allow_headers=['*'], allow_credentials=False. main.py:152-156 TrustedHostMiddleware defaults to loopback hosts and matches on host.split(':')[0], so a direct fetch to http://127.0.0.1:<port> sends Host:127.0.0.1 and passes ? it only stops DNS-rebinding via attacker domains, not direct-IP cross-origin fetches. backend/api/auth.py:125-127 resolve_tenant returns None (no auth) when edge_auth off (default), and /api/process (main.py:1355-1370) has no other auth. Mechanism is correct: wildcard CORS is load-bearing to (a) read /api/videos responses cross-origin and (b) pass the JSON-preflight for POST /api/process (main.py:1817 ? paid Gemini). /api/process 404 fast-fail (main.py:1390) needs a valid local path, but that is harvestable from the readable /api/videos index, making the cost-leak chain concrete. git log -15 -- backend/main.py shows no fix; wildcard default is live. The finding also correctly flags the internal inconsistency: bind host (main.py:174) and allowed hosts (main.py:146) both default to loopback ('widen consciously'), yet CORS alone defaults open ? an honest oversight. Proposed fix is sane and correctly scoped; not owner-gated (local-appliance secure default, no cloud-strategy/serving-go-live decision).

5. user (2026-07-02T04:45:55.977Z)

```
1       # CLAUDE.md ? project memory / agent handoff
2
3       Local, AI-powered **badminton (? racket-sports) highlight indexer + rally detector**:
4       FastAPI + SQLite + ffmpeg + GPU CV/AI, with paid **Gemini** as the cloud AI. Sibling
5       repos: `sports-data-collector`, a VLC-based `rally-annotator`.
6
7       ## Read these first
8       - **`docs/README.md`** ? doc index. **`docs/CODE_MAP.md`** ? code navigation + data
9         contracts (the cold-start map; read before touching code).
10      - **Authoritative owned-model roadmap:** `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (+
the
11        live `docs/NEXT_STEPS.md`). These SUPERSEDE the 2026-06-07 strategy docs below where
they
12        disagree (licensing, Gemma-4 stance, base model, conversational-QA).
13      - **Strategy (post-scaffolding):** `docs/COMMERCIALIZATION.md`,
`docs/PMF_MARKET_RESEARCH.md`,
14        `docs/PLATFORM_ARCHITECTURE.md`, `docs/DEFERRED_HARDENING_PLAN.md`. Also
`docs/ROADMAP.md`
15        (offline-segmenter narrative) and `docs/BACKLOG.md`.
16      - **Rally-detection quality track (design merged 2026-06-13, owner-gated, implementation
in progress):**
17        `docs/HOW_RALLY_DETECTION_WORKS.md` (2-layer mental model) ?
`docs/MULTI_SIGNAL_FUSION_PLAN.md`
18        (fuse shuttle + person + audio cues; the held-shuttle bug is the already-built
`rally_gate.py`
19        "Gate A" **not wired into production** ? cheapest win) ? `docs/EXPERIMENT_HARNESS.md`
(A/B + shadow
20        eval so cues land flag-gated/default-OFF with zero regression) ?
`docs/ROORA_LABELING_GUIDELINES.md`
21        (v8 vendor labeling spec). **`docs/NEXT_STEPS.md` has the prioritized pick-up for this
track.**
22      - **History:** `docs/archives/` ? ADRs (`decisions/DECISIONS.md`), research, and
23        **`checkpoints/`** (latest: `2026-06-strategy-foundation/`). **Archive policy:** only
move
24        **terminal-state** (DONE/REJECTED) work; live docs stay at `docs/` top level ? and
**before
25        archiving, HONESTLY update the doc first** (verify claims vs code ? flip status + a
completion
26        note ? update inbound refs ? keep the lineage ? then `git mv`), per the checklist +
living
27        lifecycle register in **`docs/DOC_STATUS.md`**.
28
29      ## Current state (June 2026)
30      - **Scaffolding complete**; strategy foundation + owned-model roadmap merged. **No
31        platform/owned-model implementation has started.** A 2026-06-10 audit-driven hardening
32        sweep landed (licensing gate, detector keying, re-ingest safety, config, eval gate,
API
33        validation, reliability, CI + test coverage) ? see git history.
34      - **Two tracks, both owner-gated:** the **committed next PLATFORM slice = async job
model**
35        (`DEFERRED_HARDENING_PLAN.md` #16, single-tenant); the **owned-model track** starts at
a
36        greenlit Phase-0 milestone (`OWNED_MODEL_IMPLEMENTATION_PLAN.md`). **Do not start
37        implementation without explicit owner approval.**
38
39      ## Architecture in one breath
40      - **Pluggable registries** (ABC + `Registry` singleton): segmenters / providers /
validators
41        / sports / annotations ? see `backend/pipeline/segmenters/base.py`. *Future:*
42        `StorageBackend` + `ComputeBackend` registries (PLATFORM §3).
43      - `video_id` = MD5 of the file ? `backend/utils/hashing.py::compute_video_id` (one
```

```
helper).
44      - Typed config = `backend/config/` (Pydantic). DB schema evolves via the `PRAGMA
45        user_version` migration ladder in `backend/infrastructure/database.py`.
46
47      ## Committed guardrails (non-negotiable)
48      - **Paid LLMs (Gemini/OpenAI/Anthropic) = inference / serving / fallback ONLY ? never a
49        training data source** (terms/copyright). Commercial-clean licensing for EVERY
dependency ?
50        code, data, AND weights: **MIT / Apache-2.0 / BSD only** (ratified 2026-06-09, owner-
agreed).
51      - **Owned-model base:** **Qwen3-VL (Apache-2.0)** primary, InternVL3 (MIT) fallback;
**Gemma-4
52        is AMBER / bench-only** (Apache grant likely but Prohibited-Use posture unconfirmed ?
counsel
53        needed; do NOT build on it without sign-off). **Reject Gemma 3** (viral license).
**Exclude
54        minors** from the training pool; per-user training data needs a separate explicit opt-
in.
55      - **v1 = rally + highlight + history + correction loop**; **gait/biometrics strictly
future**
56        (GDPR Art. 9).
57      - **Generic data schema**; `coach ? athlete` is an **optional overlay**, not baked in.
58      - **Compute = GCP ONLY (ADR-013, 2026-06-20):** GPU training + serving run on **GCP**
(`deploy/gcp/`;
59        Cloud Run+GPU for serving per ADR-012). **DigitalOcean is DROPPED for compute** (DO
GPU not enabled
60        on the account + not cleanly credit-funded; Paperspace is subscription-gated). Don't
re-evaluate
61        DO/Paperspace for compute without a new ADR; the `$200` DO credits go to **non-compute
only**. The
62        `deploy/digitalocean/*` scripts are kept solely as provider-agnostic Linux-GPU setup
reused by GCP.
63
64      ## Workflow conventions
65      - **Branch off `master`; ONE PR per work item; NO PR stacking.** Don't open a PR unless
asked.
66      - Tests: `python run_all_tests.py` (offline, fully mocked). Python 3.10+ (the `trackers`
package requires ?3.10); `ffmpeg` required.
67      - Never push to the default branch directly; descriptive commit messages.
68      - **Substantial design/project work gets a tracked project doc** following
`docs/PROJECT_DOC_TEMPLATE.md` (status header · decisions-locked · progress tracker ·
definition-of-done; archived on completion). Precedents: `docs/RALLY_QUALITY_RESEARCH.md` §7,
`docs/GOLDEN_REGRESSION_FIXTURES.md`.
69      - **Launch / "bit-flip" PRs** (a default-flip that drives a tracked project to terminal
state) MUST, in the SAME PR, run the `docs/DOC_STATUS.md` §2 archival due-diligence on that
project's docs (verify-vs-code ? flip status + completion note ? update inbound refs ? keep ADR
lineage ? `git mv` to `docs/archives/` ? record in DOC_STATUS §3), and fix any internal-link
breakage the move causes. Enforced by `.github/PULL_REQUEST_TEMPLATE.md` + the `link-check` CI
job (`python -m tools.check_md_links`).
70      - ?? Hand the heaviest machine-side checks (real video, the full GPU/WSL/WASB suite) to
the owner,
71        who has the golden corpus + labelled videos; verify pure-logic paths locally.
72
```

6. user (2026-07-02T04:46:09.546Z)

```
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
```

7. user (2026-07-02T04:46:19.443Z)

```
Preparing worktree (new branch 'fix/340-idempotent-process')
branch 'fix/340-idempotent-process' set up to track 'origin/master'.
HEAD is now at e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe
details) (#413)
```

8. user (2026-07-02T04:46:24.770Z)

CODE_MAP.md

Purpose: cold-start navigation map for the badminton-highlight-indexer ? a self-hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.

Regenerate: rerun the `build-code-map` workflow (parallel subsystem readers ? this doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure (this revision follows #40).

? **2026-06-10 audit-hardening sweep changed several behaviors documented below ? trust git history over stale lines until the next full regen:** `add_video` is now an **UPSERT** (no FK-cascade wipe on re-ingest; segmentation is destroy-AFTER-confirm via `segment_watermarks`/`prune_segments`); detector artifacts are keyed by **video_id**, not basename; **`indexing.wasb.*` is now a real typed config** (was silently dropped); `timeout_sec` defaults **1800s** (was 0/hang-forever); SQLite opens with **WAL + busy_timeout**; `match_intervals` requires real overlap (iou>0); `run_regression` baseline lives in tracked `eval_baselines/` and **exits 3 on a missing baseline**; the API validates `video_path`/`output_filename`. New tests: `test_api_endpoints`, `test_compiler`, `test_wasb_runner`, `test_reingest_safety`, `test_detector_keying`, `test_config_wasb`, `test_path_safety`, `test_reliability_hardening` (+ collector `test_licensing_gate`, `test_import_guard`, `test_cvat_robustness`).

LEGEND
- `@path` = file (always repo-relative)
- `::sym` = symbol (class/func/const) in the nearest preceding `@path`
- `->` = dataflow / call / "produces"
- `§` = data contract (canonical shape)
- `?` = gotcha / footgun
- `?` = registry / plugin extension point
- `WSL` = runs inside WSL2 conda env, NOT the Windows Python process

---

0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)

Top-level tree and the **placement rule** for each kind of file. When unsure, prefer an existing **registry / extension point** (`?`) over a new top-level file.

You're adding?	Put it in?	Notes
Backend / runtime code	`backend/<subsystem>/`	`pipeline/{segmenters,detectors,validators}`, `providers/`, `sports/`, `annotations/`, `infrastructure/`, `reporting/`, `stitcher/`, `utils/`, `config/`, `features/`, `api/`. Most additions are **register a class in the relevant `?` registry**, not a new module tree.
Eval LIBRARY (importable, pure ? metrics, calibration, a scorer, a feature)	`backend/eval/`	Pure core, no side-effects at import; CLI/I-O lives in a separate driver. These are **imported widely** (e.g. `calibrate_wasb` by 8 sites) ? they are library, not scripts; do NOT move them out.
Standalone run-driver / one-off entry CLI (has `__main__`, NOT imported by app code)	**`scripts/`**	e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`. Run as `python scripts/<name>.py`. If it needs `load_dotenv()`/`sys.path` before importing backend, add `"scripts/<name>.py" = ["E402"]` to `ruff.toml`.
Ops / maintenance tool (not part of the served app ? cache GC, batch admin)	`tools/`	Run as `python -m tools.<name>` (it's an importable package). e.g. `tools/cleanup_caches.py`. Keep the destructive decision path pure + unit-tested.
Training code (model fitting)	`training/`	Behind the CI-enforced import wall (ADR-008) ? `backend.main` must not import it. Own `requirements-train.txt`.
A test	`tests/test_*.py`	pytest auto-discovers; the full-suite lane has a **70% coverage floor**. Pure/offline + mocked (no GPU/WSL/network).
A doc (design, plan, living log)	`docs/`	Index it in `docs/README.md`. Only **terminal-state** (DONE/REJECTED) work moves to `docs/archives/`.

A throwaway repro / debug script	`scratch/`	**Gitignored** + excluded from ruff/pytest. Never imported by app code.
A model artifact (weights + manifest)	`artifacts/<name>/<ver>/`	ADR-008: **manifest committed** (provenance), **weights gitignored**.
Eval baseline / leaderboard JSON	`eval_baselines/`	Tracked (survives a clean checkout); the no-regression gate reads it.
Committed regression fixture (video-free golden)	`eval_baselines/fixtures/<slug>/`	Tracked, **LF-pinned** (`.gitattributes`). Synthetic Tier-0 (or owner-cleared, de-attributed real). Drives the `golden_fixtures` characterization gate with **no video/GPU/DB**. See `docs/GOLDEN_REGRESSION_FIXTURES.md`; mint via `python -m backend.eval.golden_fixtures --freeze-synthetic` / `--freeze-real` (refusal-gated).
Pipeline run outputs (reels, trajectories, DBs, frames)	`output/`	**Gitignored.** Per-video isolation is the tracked **`PER_VIDEO_WORKSPACE.md`** project (P0 helper shipped #264; supersedes the archived `PER_VIDEO_OUTPUT_LAYOUT`).

Canonical files that stay at the repo ROOT (referenced by CI / packaging / docs ? do not move):
`pyproject.toml` (PEP 621 packaging, hatchling; `indexer` entry point) · `run_all_tests.py` (CI: `ci.yml`) ·
`run_regression.py` (the no-regression gate CLI, cited in `EXPERIMENT_HARNESS.md`) · `run_eval.py`
(grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic `setup.py` moved to
`scripts/doctor.py` (it shadowed the build system); invoke via `python scripts/doctor.py` or `indexer --setup`.
New standalone run-drivers go in **`scripts/`**, not the root.

---

1. PIPELINE

1A. Runtime: video file -> highlight reel

```
typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-defaults, never fatal)
                                          @backend/config/loader.py::load_config -> @backend/config/models.py::AppConfig
  -> video_id = MD5(whole file)           @backend/utils/hashing.py::compute_video_id  (single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint copies)
  -> validate @backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
       FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity -> SceneCut -> Blur -> Relevance   (? ACTUAL registration/exec order ? see @backend/pipeline/validators/__init__.py)
       [validation FAIL -> persist status="failed", return early; report STILL emitted in finally]
  -> db.add_video(video_id, filepath, status, [r.dict()])   @backend/infrastructure/database.py
  -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback 'motion')
  -> seg = segmenter_registry.get(seg_name)(db, global_config)   ? @backend/pipeline/segmenters/base.py::segmenter_registry  (400 + available list if missing)
  -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
       [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here without stitching]
  -> finally: emit_indexer_report(...)    @backend/reporting/__init__.py  (? ALWAYS runs ? even on validation failure / exception; NEVER raises; grafts response["reports"])
  -> select segment ids -> [(start,end)] (+ stitching padding)
  -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name) @backend/pipeline/stitcher/compiler.py
       per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy -> output/<name>.mp4
```

Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same shape):

```
P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
```

```
 -> P2 candidate "action windows" from motion/trajectory  [STOP: candidates persisted via
db.add_candidate_segment]
 -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor -> provider.analyze_video(clip,
prompt)   ? provider_registry
 -> P4 db.add_play_segment + db.link_candidate_to_segment
```
Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate (clamp/dedup) ->
add_play_segment.
Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground truth).

1B. WASB shuttle substrate (P2 of wasb_hybrid)
```

**9. user (2026-07-02T04:46:25.897Z)**

```
1100   _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
1101   # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
1102   # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
1103   provenance = {
1104   "path": "cloud_run",
1105   "requested": req.compute,
1106   "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
1107   "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
1108   "execution": result.execution,
1109   "outputs_uri": result.outputs_uri,
1110   }
1111   logger.info(
1112   "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
1113   result.execution,
1114   provenance["job"],
1115   provenance["region"],
1116   result.outputs_uri,
1117   video_id,
1118   )
1119   return (
1120   {
1121   "video_id": video_id,
1122   "status": "success",
1123   "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
1124   "results": [r.model_dump() for r in val_results],
1125   "play_segments": segments,
1126   "compute": provenance,
1127   },
1128   True,
1129   )
1130
1131
1132   def _execute_processing(
1133   config=None, db=None, req=None, progress=None
1134   ) -> Dict[str, Any]:
1135   """The full hash ? validate ? segment ? report pipeline for one video.
1136
1137   Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
1138   module-level ``global_config``/``db_instance`` (backward-compatible with existing
1139   tests that pass only ``req``). Route handlers always pass explicit params; older
1140   test code that calls ``_execute_processing(req)`` still works.
1141
1142   ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
1143   Raises on unexpected internal errors ? the caller records those as a job failure
1144   """
```
```

```
        # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
        # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
        if config is not None and not isinstance(config, AppConfig):
            # Old signature: _execute_processing(req, progress=None)
            if req is None and db is not None and isinstance(db, ProcessRequest):
                req = db
                db = None
            elif req is None and isinstance(config, ProcessRequest):
                req = config
                config = None
        # Resolve config/db from module globals when not passed explicitly
        if config is None:
            config = global_config
        if db is None:
            db = db_instance

        notify = progress or (lambda stage: None)

        notify("hashing")
        # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
        # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
        # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
        # consumers (hash / validators / segmenter) use the resolved local path.
        local_video = storage.acquire_local_input(
            req.video_path, getattr(config, "storage", None)
        )
        video_id = compute_video_id(local_video)
        was_reingest = db.get_video(video_id) is not None

        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
        notify("validating")
        logger.info(f"Running validations for {req.video_path}...")
        val_results, val_context = registry.run_all_with_context(local_video, config)
        failures = [r for r in val_results if not r.passed]

        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
        default_seg = getattr(
            getattr(config, "indexing", None), "default_segmenter", "motion"
        )
        seg_name = req.segmenter or default_seg
        segmenter_actual = None
        segmentation_ran = False
        # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
        # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
        # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
        compute_actual: Optional[str] = None
        response: Optional[Dict[str, Any]] = None
        try:
            if failures:
                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
                # status; it no longer destroys the video's prior good segments.
                db.add_video(
                    video_id,
                    req.video_path,
                    "failed",
                    [r.model_dump() for r in val_results],
                )
```

```
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": "Video failed validation checks.",
                    "results": [r.model_dump() for r in val_results],
                }
                return response

            # 2. Run segmenter & indexer
            logger.info("[API] Video passed checks. Segmenting and indexing...")
            db.add_video(
                video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
            )

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                # Submit-time validation already 400s unknown names; this is the defensive
                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
                available = list(segmenter_registry.list_available().keys())
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            logger.info(f"[API] Using segmenter: {seg_name}")
            notify(f"segmenting ({seg_name})")
            segmenter_actual = seg_name
            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
            play_wm, cand_wm = db.segment_watermarks(video_id)
            # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
            # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
            # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
            # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
            # they're dropped on the cloud path in this slice ? a documented follow-up.)
            remote = _maybe_dispatch_remote(
                config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
            )
            if remote is not None:
                response, segmentation_ran = (
                    remote  # `ran` = did the cloud job actually execute (report)
                )
                return response  # the cloud branch already stamped response["compute"]
(path=cloud_run)
            # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
            compute_actual = "in_process"
            segmenter = segmenter_cls(db, config)
            try:
                result = segmenter.process_video(
                    video_id, local_video, req.player_pool, req.max_frames
                )
            except Exception:
```

```
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
                db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result
            notify("finalizing")
            _finalize_reingest(db, video_id, segments, play_wm, cand_wm)

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
            # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
            # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
            # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
            if isinstance(response, dict):
                response.setdefault(
                    "compute",
                    {
                        "path": compute_actual or "not_run",
                        "requested": getattr(req, "compute", None),
                    },
                )
            prov = response.get("compute", {}) if isinstance(response, dict) else {}
            logger.info(
                "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
                video_id,
                getattr(req, "compute", None),
                prov.get("path"),
                prov.get("dispatched"),
                (response or {}).get("status") if isinstance(response, dict) else None,
            )
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
            # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
            # doesn't surface arbitrary run signals.)
```

```
1314            paths = emit_indexer_report(
1315                db=db,
1316                video_id=video_id,
1317                # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
1318                # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
1319                # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
1320                # still preserved in DB provenance via db.add_video.
1321                video_path=local_video,
1322                config=config,
1323                val_results=val_results,
1324                val_context=val_context,
1325                segmenter_requested=req.segmenter,
1326                segmenter_actual=segmenter_actual,
1327                was_reingest=was_reingest,
1328                segmentation_ran=segmentation_ran,
1329            )
1330            if paths and isinstance(response, dict):
1331                response["reports"] = paths
1332
1333
1334    def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
1335        """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
1336
1337        Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
that needs a
1338        missing cloud SDK, the network, or a credential returns ``None`` so the free-space
guard degrades
1339        to a no-op rather than blocking a job it cannot measure."""
1340        try:
1341            sc = (
1342                storage_config.model_dump()
1343                if hasattr(storage_config, "model_dump")
1344                else (storage_config or {})
1345            )
1346            backend = storage.get_storage(input_ref.backend, config=sc)
1347            return int(backend.stat(input_ref).size)
1348        except Exception as e:  # noqa: BLE001 - measuring is best-effort; inability must
not block
1349            logger.debug(
1350                "free-space: could not stat remote source %s: %s", input_ref.uri, e
1351            )
1352            return None
1353
1354
1355    @app.post("/api/process", status_code=202)
1356    def process_video(req: ProcessRequest, request: Request):
1357        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
1358
1359        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
```

10. user (2026-07-02T04:46:34.337Z)

```
280     global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
281     # P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if
it were
282     # GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
283     _instance_lock: Optional[Any] = None
284
285     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
286     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
```

```
    # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
    # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
    # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
    # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
    # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
    from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable
on backend.main)
        _MAX_DRAIN_WAKE_SEC,
        JobWaiting,
        _arm_wake_timer,
        _drain_due_jobs,
        _get_executor,
        _job_to_request,
        _maybe_record_job_cost,
        _run_claimed_job,
        _schedule_next_drain_if_waiting,
    )


    # Legacy thin wrapper for any remaining direct calls during transition.
    # New code should import load_app_config / AppConfig from backend.config directly.
    def load_config(config_path: str = "config.json") -> Dict[str, Any]:
        cfg = load_app_config(config_path)
        return cfg.model_dump(mode="json")


    # --- API Models ---
    class ProcessRequest(BaseModel):
        video_path: str
        player_pool: Optional[List[str]] = None
        max_frames: Optional[int] = None
        segmenter: Optional[str] = None
        # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
        # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
        locale: Optional[str] = None
        # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
        # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
        # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
        compute: Optional[str] = None


    class AnnotationsRequest(BaseModel):
        # Finished rally labels for a video, as the annotator's byte-compatible CSV (P8,
from the Studio
        # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is
written verbatim.
        video_id: str
        csv: str


    class AssetRequest(BaseModel):
        # "Store this video in my medium" (STUDIO_MEDIA_LIFECYCLE.md P4, D14). Exactly ONE
source:
        #   uploaded_path ? a local file from the chunked upload (the durable copy's source;
kept as the
        #                   local working copy for the run), OR
        #   source_uri    ? a video ALREADY at rest in a medium (a pasted
s3:///gcs:///gdrive:// URI) ?
        #                   registered as-is, no copy (D14b).
```

```
        # medium_uri is the destination store (a bucket/Drive/local folder URI). The asset
is keyed by
        # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content
hash.
        uploaded_path: Optional[str] = None
        source_uri: Optional[str] = None
        medium_uri: str
        sport: Optional[str] = None


    class QueryRequest(BaseModel):
        video_id: str
        server: Optional[str] = None
        receiver: Optional[str] = None
        duration_min: Optional[float] = None
        event_type: Optional[str] = None


    class CompileRequest(BaseModel):
        video_id: str
        segment_ids: List[int]
        output_filename: str
        # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
        # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).
        locale: Optional[str] = None


    def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.

        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
        keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
        """
        if segments:
            db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
        else:
            db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)


    # --- API Endpoints ---
    @app.get("/api/config")
    def get_config(request: Request):
        # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
field. Today
        # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
id), so this
        # is byte-identical for the real fields ? but it fails-closed if a secret is ever
added (guarded
        # by tests/test_redact.py + the /api/config redaction test).
        cfg = request.app.state.config
        if cfg is None:
            return {}
        return redact_secrets(cfg.model_dump(mode="json"))


    @app.get("/api/health")
    def health(request: Request):
        """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
        (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
        confirm the engine is reachable and which detector is wired. Never raises."""
```

```
396         cfg = request.app.state.config
397         sport = getattr(cfg, "sport", None)
398         indexing = getattr(cfg, "indexing", None)
399         return {
400             "status": "ok",
401             "sport": sport,
402             "default_segmenter": getattr(indexing, "default_segmenter", None),
403         }
404
405
406     # P1: the bundled SPA pins this ? the contract version of the /api surface. BUMP when an
existing
407     # endpoint's request/response shape changes incompatibly (purely additive endpoints need
no bump).
408     API_VERSION = 1
409
410
411     def _engine_version() -> str:
412         """Installed package version (wheel/editable); 'unknown' if not importable as a
distribution."""
413         try:
414             return _pkg_version("badminton-highlight-indexer")
415         except PackageNotFoundError:
416             return "unknown"
417
418
419     @app.get("/api/version")
420     def api_version():
421         """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md
P1)."""
422         return {
423             "engine_version": _engine_version(),
424             "api_version": API_VERSION,
425             "ui": _bundled_ui_version(),
426         }
427
428
429     @app.get("/api/capabilities")
```

11. user (2026-07-02T04:46:35.718Z)

```
1355     @app.post("/api/process", status_code=202)
1356     def process_video(req: ProcessRequest, request: Request):
1357         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
1358
1359         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
1360         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
1361         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
1362         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
1363         """
1364         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
1365         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
1366         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
1367         try:
1368             owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
1369         except edge_auth.EdgeAuthError as e:
1370             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
1371
1372         allowed_root = getattr(
1373             getattr(request.app.state.config, "security", None), "allowed_video_root", None
1374         )
1375         try:
```

```
            validate_input_path(req.video_path, allowed_root=allowed_root)
            # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
            # scheme, which we map to a clean 400 (an unknown scheme must never 500).
            input_ref = storage.resolve_input_ref(
                req.video_path, getattr(request.app.state.config, "storage", None)
            )
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
        # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
        # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
        # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
        # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
        # rejects a non-local URI as out-of-root.
        if input_ref.backend == "local" and not os.path.exists(input_ref.key):
            raise HTTPException(
                status_code=404, detail=f"Video file not found at '{req.video_path}'"
            )
        # P2 (D11): a bucket/Drive source is fetched to a scratch copy on the worker; fail
fast with 507
        # if this box can't hold that copy. Best-effort ? an unstattable remote
(network/SDK) or an
        # unreadable temp fs skips the guard (measuring must not deny a legit job); the
fetch itself
        # still fails loudly downstream if space genuinely runs out. Local inputs need no
copy ? skipped.
        if input_ref.backend != "local":
            need = _remote_input_size_bytes(
                input_ref, getattr(request.app.state.config, "storage", None)
            )
            require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
                status_code=400,
                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
            )

        job_id = uuid.uuid4().hex
        if not request.app.state.db.create_job(
            job_id,
            req.video_path,
            req.segmenter,
            req.player_pool,
            req.max_frames,
            owner_id=owner_id,
            locale=req.locale,
            compute=req.compute,
        ):
            raise HTTPException(status_code=500, detail="Failed to persist job record.")
        _get_executor().submit(
            _drain_due_jobs, request.app.state.config, request.app.state.db
        )
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )

```

```
1430
1431    @app.get("/api/jobs/{job_id}/cost")
1432    def get_job_cost(job_id: str, request: Request):
1433        """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
1434        GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
1435        null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
1436        cost = request.app.state.db.get_job_cost(job_id)
1437        if cost is None:
1438            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
1439        fx = request.app.state.config.billing.fx_inr_per_usd
1440        usd = cost.get("cost_usd")
1441        cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
1442        cost["fx_inr_per_usd"] = fx
1443        return cost
1444
1445
1446    @app.get("/api/videos/{video_id}/cost")
1447    def get_video_cost(video_id: str, request: Request):
1448        """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the
1449        per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
several."""
1450        agg = request.app.state.db.get_video_cost(video_id)
1451        fx = request.app.state.config.billing.fx_inr_per_usd
1452        agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
1453        agg["fx_inr_per_usd"] = fx
1454        return agg
1455
1456
1457    @app.get("/api/jobs/{job_id}")
1458    def get_job(job_id: str, request: Request):
1459        """Job state: {status, progress, result, reports}. `status` = execution state
1460        (queued|running|done|failed); the pipeline verdict is result["status"]."""
1461        job = request.app.state.db.get_job(job_id)
1462        if not job:
1463            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
1464        result = job.get("result")
1465        return {
1466            "job_id": job["id"],
1467            "status": job["status"],
1468            "progress": job.get("progress"),
1469            "video_id": job.get("video_id"),
1470            "error": job.get("error"),
1471            "result": result,
1472            "reports": (result or {}).get("reports"),
1473            "created_at": job.get("created_at"),
1474            "started_at": job.get("started_at"),
1475            "finished_at": job.get("finished_at"),
1476        }
1477
1478
1479    # --- Chunked upload (B2, PR-2) ----------------------------------------------------
1480    # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
1481    # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
1482    # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
1483    # sequential-part logic, and abort cleanup ? is unchanged.
1484    from backend.api import uploads as uploads_api
```

12. user (2026-07-02T04:46:45.829Z)

```
1       """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
```

```
P3).

      Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
      The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
      that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
      re-claimed when due ? with no central scheduler (the DB is the clock).

      The monkeypatch contract these functions rely on (and that tests depend on):
      every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
      `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
      `backend.main` module object at call-time (`import backend.main as _main`), never as a
      bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
      ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
      `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
      inside the functions also avoids the circular import at module load (main.py imports
this
      module; this module must not import main at import time).
      """

      import logging
      import threading
      import traceback
      from concurrent.futures import ThreadPoolExecutor
      from typing import Any, Dict, Optional

      logger = logging.getLogger(__name__)

      # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
      # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
      # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
      # already parallelize their AI handoff internally via their own pools.
      #
      # ? O2 RISK (CODE_CLEANUP_AUDIT §3c): a single worker means ONE stuck or hung job
      # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
      # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
      #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
      #   - detector timeout_sec kills hung WSL/GPU calls
      #   - Cloud Run poll has a max_wait_sec bound
      # A future slice should add a per-job wall-clock timeout (the executor itself
      # cannot enforce timeouts on the Thread).
      _job_executor: Optional[ThreadPoolExecutor] = None


      def _get_executor() -> ThreadPoolExecutor:
          """Lazy module-global executor; tests monkeypatch this for inline execution."""
          global _job_executor
          if _job_executor is None:
              _job_executor = ThreadPoolExecutor(
                  max_workers=1, thread_name_prefix="jobworker"
              )
          return _job_executor


      class JobWaiting(Exception):
          """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
          WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
          NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
          releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
          is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.

          The wiring is additive: the production segmenter path never raises this today (zero
```

```
        behaviour change), so a job runs exactly as before. The hook is what a later slice
        (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
        """

        def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
            super().__init__(f"job parked for {delay_sec:.0f}s")
            self.delay_sec = max(0.0, float(delay_sec))
            self.progress = progress


    def _job_to_request(job: Dict[str, Any]):
        """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
        re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
        the original submit. The row stores exactly the ProcessRequest fields."""
        import backend.main as _main

        return _main.ProcessRequest(
            video_path=job["video_path"],
            player_pool=job.get("player_pool"),
            max_frames=job.get("max_frames"),
            segmenter=job.get("segmenter"),
            # v8 compute-picker: the worker runs _execute_processing ?
_maybe_dispatch_remote, which
            # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
only read off
            # the job row for analytics). NULL ? server default = pre-picker behaviour.
            compute=job.get("compute"),
        )


    def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
        """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
        terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).

        Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
        the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
        result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
        job self-scheduled and will be re-claimed when `next_poll_at` is due.
        """
        import backend.main as _main

        job_id = job["id"]
        # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
        # original /api/process pipeline). Both record their terminal state the same way;
only process
        # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
        is_compile = job.get("kind") == "compile"
        try:
            if is_compile:
                result = _main._execute_compile(
                    config,
                    db,
                    job,
                    progress=lambda stage: db.set_job_progress(job_id, stage),
                )
            else:
                req = _main._job_to_request(job)
                result = _main._execute_processing(
                    config,
                    db,
```

```
                    req,
                    progress=lambda stage: db.set_job_progress(job_id, stage),
                )
            if result.get("video_id"):
                db.set_job_video_id(job_id, result["video_id"])
            db.mark_job_done(job_id, result)
            if not is_compile:
                _maybe_record_job_cost(
                    job, result, config, db
                )  # M8 cost ledger (default-OFF; process jobs only)
                _maybe_run_flywheel(
                    job, result, config
                )  # M11 data-flywheel (process jobs only)
        except _main.JobWaiting as w:
            # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
            logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
            db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
        except Exception as e:
            logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
            db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")


    def _maybe_record_job_cost(
        job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
    ) -> None:
        """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
        ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
        (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
        persist it.

        ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
        ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
        FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
        + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
        With the placeholder pricebook the cost is 0 regardless.

        Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
        claimed row (carries ``owner_id``); ``result`` is the pipeline output."""

        billing_cfg = getattr(config, "billing", None)
        if not billing_cfg or not getattr(billing_cfg, "enabled", False):
            return
        try:
            usage = (result or {}).get("usage") or {}
            db.record_job_cost(
                job["id"],
                usage=usage,
                pricebook_version=billing_cfg.pricebook_version,
                gpu_type=usage.get("gpu_type"),
                compute_backend=(result or {}).get("compute_backend"),
                owner_id=job.get("owner_id"),
                margin=billing_cfg.margin,
            )
            logger.info(
                "Recorded cost for job %s (pricebook=%s).",
                job["id"],
                billing_cfg.pricebook_version,
            )
        except Exception as e:  # never wedge a completed job on bookkeeping
```

```
            logger.warning(
                "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
            )


    def _maybe_run_flywheel(
        job: Dict[str, Any], result: Dict[str, Any], config: Any
    ) -> None:
        """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
        ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
        (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
        land), so this captures **nothing** in production. When activated it captures the
consented job's
        artifacts into the per-video workspace (? later labeling ? the golden training set).

        Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
        row (carries ``owner_id``); ``result`` is the pipeline output."""

        flywheel_cfg = getattr(config, "flywheel", None)
        if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
            return
        try:
            from backend import flywheel

            flywheel.run_for_job(job, result, config)
        except Exception as e:  # never wedge a completed job on the flywheel
            logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)


    def _drain_due_jobs(config: Any, db: Any) -> int:
        """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
        `next_poll_at` is due), earliest-due first, until the table has nothing runnable.

        Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
        up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
        a single-worker box keeps making progress with no central timer (the DB is the
clock).
        Returns the number of jobs that reached a terminal/parked state this tick.
        """
        import backend.main as _main

        ran = 0
        while True:
            job = db.claim_due_job()
            if job is None:
                break
            ran += 1
            _main._run_claimed_job(job, config, db)
        # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
        # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
        # on the next submit/drain. Best-effort ? never raises into the worker.
        _main._schedule_next_drain_if_waiting(config, db)
        return ran


    def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
```

```
231         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
232         self-scheduled job resumes with no further client submit. The drain itself re-checks
233         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
234         import backend.main as _main
235
236         try:
237             delay = db.seconds_until_next_due()
238         except Exception as e:  # pragma: no cover - defensive; never wedge the worker
239             logger.error(f"Could not compute next-due delay: {e}")
240             return
241         if delay is None:
242             return  # nothing waiting
243         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
244         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)
245
246
247     def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
248         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
249         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
250         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
251         import backend.main as _main
252
253         timer = threading.Timer(
254             delay_sec,
255             lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
256         )
257         timer.daemon = True
258         timer.start()
259
260
261     #: A parked job further out than this is re-checked by a capped wake rather than one
long
262     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
263     _MAX_DRAIN_WAKE_SEC = 60.0
264
```

13. user (2026-07-02T04:46:55.873Z)

```
83:         ``user_version`` stays un-bumped ? and the next open would re-run the same
102:             # Versioned migrations live here. PRAGMA user_version is the schema
105:             version = cursor.execute("PRAGMA user_version").fetchone()[0]
148:             # future: if version < 10: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 10
258:         cursor.execute("PRAGMA user_version = 1")
285:         cursor.execute("PRAGMA user_version = 2")
301:         cursor.execute("PRAGMA user_version = 3")
327:         cursor.execute("PRAGMA user_version = 4")
365:             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
375:         cursor.execute("PRAGMA user_version = 5")
386:             ("compute_backend", "TEXT"),  # local_gpu | cloud_run | ...
393:             ("cost_usd", "REAL"),  # computed total (USD = source of truth)
423:         # Seed a PLACEHOLDER version with every rate 0.0 ? cost_usd computes to 0 (nothing
437:         cursor.execute("PRAGMA user_version = 6")
446:         cursor.execute("PRAGMA user_version = 7")
449:         # v8: record the per-request COMPUTE choice the SPA compute-picker sent ("local"
forces the
454:         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN compute TEXT")
455:         cursor.execute("PRAGMA user_version = 8")
465:         cursor.execute("PRAGMA user_version = 9")
669:     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
```

```
764:    def create_job(
774:        compute: Optional[str] = None,
780:        ``compute`` (v8) is the SPA compute-picker choice ("local"/"cloud") the worker
reconstructs +
784:            "owner_id, locale, compute) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?, ?, ?)",
794:                compute,
907:        An unknown version ? ``{}`` (so every cost computes to 0.0)."""
939:        compute_backend: Optional[str] = None,
960:        cost_usd, breakdown = billing.compute_cost_usd(usage, rates)
971:            "UPDATE jobs SET owner_id=?, compute_backend=?, gpu_type=?,
gpu_active_seconds=?, "
976:                compute_backend,
1004:                    "SELECT id, video_id, owner_id, compute_backend, gpu_type,
gpu_active_seconds, "
1018:    def get_video_cost(self, video_id: str) -> Dict[str, Any]:
1063:        computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
1091:            logger.error(f"Failed to compute next-due delay: {e}")
1094:    def claim_due_job(self) -> Optional[Dict[str, Any]]:
```

14. user (2026-07-02T04:47:02.691Z)

```
95          def _init_db(self):
96              """Initializes tables for videos, play_segments, and events."""
97              with self._connect() as conn:
98                  cursor = conn.cursor()
99
100                 self._create_base_tables(cursor)
101
102                 # Versioned migrations live here. PRAGMA user_version is the schema
103                 # contract ? bump it for every change and add an ordered `if version < N`
104                 # block below. Tests assert the expected version, so accidental drift fails
CI.
105                 version = cursor.execute("PRAGMA user_version").fetchone()[0]
106
107                 # P2d upgrade-safety (no-op for a fresh DB or one already at
SCHEMA_VERSION):
108                 # (1) DOWNGRADE GUARD ? a DB from a NEWER app (version > what we know) has
schema this
109                 #     binary can't reason about; opening it could corrupt data, so refuse
loudly.
110                 if version > SCHEMA_VERSION:
111                     raise SchemaTooNewError(
112                         f"This database (schema v{version}) was created by a newer version
of the app "
113                         f"than this one (knows up to v{SCHEMA_VERSION}). Upgrade the app, or
point it at "
114                         f"a different data directory (RALLY_APP_HOME). DB: {self.db_path}"
115                     )
116                 # (2) BACKUP-BEFORE-MIGRATE ? an EXISTING older DB is about to be upgraded
in place; a
117                 #     failed migration could corrupt it, so snapshot it first (best-effort:
WARN, never
118                 #     block the upgrade). version 0 = a brand-new DB ? nothing to back up.
119                 if 0 < version < SCHEMA_VERSION:
120                     self._backup_before_migrate(conn, version)
121
122                 if version < 1:
123                     self._migrate_to_v1(cursor)
124
125                 if version < 2:
126                     self._migrate_to_v2(cursor)
127
128                 if version < 3:
129                     self._migrate_to_v3(cursor)
```

```
130
131                 if version < 4:
132                     self._migrate_to_v4(cursor)
133
134                 if version < 5:
135                     self._migrate_to_v5(cursor)
136
137                 if version < 6:
138                     self._migrate_to_v6(cursor)
139
140                 if version < 7:
141                     self._migrate_to_v7(cursor)
142
143                 if version < 8:
144                     self._migrate_to_v8(cursor)
145
146                 if version < 9:
147                     self._migrate_to_v9(cursor)
148                 # future: if version < 10: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 10
149
150                 conn.commit()
151
152         def _backup_before_migrate(
153             self, conn: sqlite3.Connection, from_version: int
154         ) -> None:
155             """Snapshot the DB to ``<db>.bak-v<from_version>`` before an upgrade migration
(P2d).
156
157             Checkpoints the WAL into the main file first so the copy is complete. Best-
effort: a backup
158             failure WARNS but does NOT block the upgrade (the ladder is itself crash-safe;
refusing to
159             start because a backup couldn't be written would be worse than proceeding
without one)."""
160             backup = f"{self.db_path}.bak-v{from_version}"
161             try:
162                 conn.execute("PRAGMA wal_checkpoint(FULL)")
163                 shutil.copy2(self.db_path, backup)
164                 logger.info(
165                     "[DB] backed up v%s database before upgrading ? %s",
166                     from_version,
167                     backup,
168                 )
169             except Exception as exc:  # noqa: BLE001 ? the backup is a best-effort safety
net; its failure
170                 # must NEVER block the in-place upgrade (the migration ladder is itself
crash-safe). Log loud.
171                 logger.warning(
172                     "[DB] could not back up the database before migrating from v%s (%s); "
173                     "proceeding with the in-place upgrade.",
174                     from_version,
```

15. user (2026-07-02T04:47:03.727Z)

```
438
439         def _migrate_to_v7(self, cursor):
440             # v7: record the UI LOCALE a job was submitted in (the language the SPA was
rendered in,
441             # e.g. "kn", "hi-IN"). NULLABLE ? NULL = unrecorded = byte-identical to today
(CLI / older
442             # clients / pre-v7 rows). Enables PMF analytics ("which language markets use the
tool",
443             # count_jobs_by_locale) and pairs with the localized reel watermark (the SPA
sends the same
```

```
            # locale to /api/compile). Additive-only ALTER; idempotent so an interrupted
upgrade re-runs.
            self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN locale TEXT")
            cursor.execute("PRAGMA user_version = 7")

        def _migrate_to_v8(self, cursor):
            # v8: record the per-request COMPUTE choice the SPA compute-picker sent ("local"
forces the
            # in-process segmenter; "cloud" forces a Cloud Run dispatch). It must survive
the async
            # submit?worker boundary ? the worker reconstructs the ProcessRequest from this
row, so a
            # NULL/unset value (CLI / older clients / pre-v8 rows) means "use the server
default", i.e.
            # byte-identical to today. Additive-only ALTER; idempotent so an interrupted
upgrade re-runs.
            self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN compute TEXT")
            cursor.execute("PRAGMA user_version = 8")

        def _migrate_to_v9(self, cursor):
            # v9: the jobs table now holds TWO kinds of work ? `kind`='process' (the
original, NULL ?
            # process for pre-v9 rows) and `kind`='compile' (the async reel stitch, #329;
/api/compile
            # is now 202+poll like /api/process so a slow ffmpeg stitch can't trip the edge
524). `params`
            # is the kind-specific JSON payload a compile job carries
(segment_ids/output_filename/locale)
            # since those don't map onto the process columns. Additive-only ALTERs;
idempotent.
            self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN kind TEXT")
            self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN params TEXT")
            cursor.execute("PRAGMA user_version = 9")

        def add_video(
            self,
            video_id: str,
            filepath: str,
            status: str,
            validation_results: List[Dict[str, Any]],
        ) -> bool:
            """Register or update a video row WITHOUT touching its child segments.

            Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
            with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
            That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
            before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
            row in place; clearing segments is now an explicit, deliberate operation
            (clear_video_data / prune_segments).
            """
            return self._execute_write(
                "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
?, ?) "
                "ON CONFLICT(id) DO UPDATE SET "
                "filepath=excluded.filepath, status=excluded.status, "
                "validation_results=excluded.validation_results",
                (video_id, filepath, status, json.dumps(validation_results)),
                "Failed to add video",
            )

        def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
            """Return (max play_segment id, max candidate_segment id) for a video (0 if
```

```
none).

            Captured before a (re)segmentation so the run's new rows (id > watermark) can be
            told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
            """
```

16. user (2026-07-02T04:47:18.422Z)

```
        def query_candidate_segments(
            self,
            video_id: str,
            detector: Optional[str] = None,
            only_unlabeled: bool = False,
        ) -> List[Dict[str, Any]]:
            """Fetch candidate detections for the annotation / fine-tuning workflow.
            `stats` and `detection_params` are deserialized from JSON."""
            query = "SELECT * FROM candidate_segments WHERE video_id = ?"
            params: List[Any] = [video_id]

            if detector:
                query += " AND detector = ?"
                params.append(detector)
            if only_unlabeled:
                query += " AND human_label IS NULL"

            query += " ORDER BY start_time"

            candidates = []
            with self._connect() as conn:
                rows = conn.cursor().execute(query, params).fetchall()
                for row in rows:
                    d = dict(row)
                    d["stats"] = json.loads(d["stats"] or "{}")
                    d["detection_params"] = json.loads(d["detection_params"] or "{}")
                    candidates.append(d)
            return candidates

        def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
            with self._connect() as conn:
                row = (
                    conn.cursor()
                    .execute("SELECT * FROM videos WHERE id = ?", (video_id,))
                    .fetchone()
                )
                if row:
                    d = dict(row)
                    d["validation_results"] = json.loads(d["validation_results"] or "[]")
                    return d
            return None

        def list_videos(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
            """All indexed videos, most-recently-added first. Orders by rowid (a monotonic
insertion
            counter, since id is a TEXT PK) rather than the 1-second-granularity
processed_at, which
            would tie within a batch. Mirrors get_video's row shape (validation_results
parsed). Powers
            GET /api/videos for the operator console (PACKAGING_PLAN.md P1)."""
            sql = "SELECT * FROM videos ORDER BY rowid DESC"
            params: tuple = ()
            if limit is not None:
                sql += " LIMIT ?"
                params = (int(limit),)
            with self._connect() as conn:
                rows = conn.cursor().execute(sql, params).fetchall()
```

```
            out: List[Dict[str, Any]] = []
            for row in rows:
                d = dict(row)
                d["validation_results"] = json.loads(d["validation_results"] or "[]")
                out.append(d)
            return out

        def query_play_segments(
            self, video_id: str, filters: Dict[str, Any]
        ) -> List[Dict[str, Any]]:
            """
            Dynamically queries play segments based on filters:
            - duration_min: float
            - server: string
            - receiver: string
            - event_type: string (e.g. smash, foul)
            """
            query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
            params = [video_id]

            if filters.get("duration_min"):
                query += " AND r.duration >= ?"
                params.append(filters["duration_min"])

            if filters.get("server"):
                query += " AND r.server = ?"
                params.append(filters["server"])

            if filters.get("receiver"):
                query += " AND r.receiver = ?"
                params.append(filters["receiver"])

            if filters.get("event_type"):
                # Check if there is an event of a specific type inside this segment
                query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
                params.append(filters["event_type"])

            segments_list = []
            with self._connect() as conn:
                cursor = conn.cursor()
                rows = cursor.execute(query, params).fetchall()

                for row in rows:
                    r_dict = dict(row)
                    r_id = r_dict["id"]
                    r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")

                    # Fetch associated events
                    event_rows = cursor.execute(
                        "SELECT * FROM events WHERE segment_id = ?", (r_id,)
                    ).fetchall()
                    r_dict["events"] = []
                    for er in event_rows:
                        e_dict = dict(er)
                        e_dict["details"] = json.loads(e_dict["details"] or "{}")
                        r_dict["events"].append(e_dict)

                    segments_list.append(r_dict)

            return segments_list

        def clear_video_data(self, video_id: str):
            with self._connect() as conn:
                conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
```

```
758                 conn.commit()
759
760         # --- Async job store (DEFERRED_HARDENING_PLAN #16) ------------------------------
761         # Writers follow the repo convention (swallow + logger.error, return bool); the
762         # worker logs loudly on a failed transition so a job can't silently wedge.
763
764         def create_job(
765             self,
766             job_id: str,
767             video_path: str,
768             segmenter: Optional[str] = None,
769             player_pool: Optional[List[str]] = None,
770             max_frames: Optional[int] = None,
771             policy: Optional[Dict[str, Any]] = None,
772             owner_id: Optional[str] = None,
773             locale: Optional[str] = None,
774             compute: Optional[str] = None,
775         ) -> bool:
776             """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
777             (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
``owner_id``
778             (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
(default).
779             ``locale`` (v7) is the UI language the submission came from ? NULL when
unrecorded.
780             ``compute`` (v8) is the SPA compute-picker choice ("local"/"cloud") the worker
reconstructs +
781             honours; NULL ? the server default (byte-identical to pre-picker behaviour)."""
782             return self._execute_write(
783                 "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy, "
784                 "owner_id, locale, compute) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?, ?, ?)",
785                 (
786                     job_id,
787                     video_path,
788                     segmenter,
789                     json.dumps(player_pool) if player_pool is not None else None,
790                     max_frames,
791                     json.dumps(policy) if policy is not None else None,
792                     owner_id,
793                     locale,
794                     compute,
795                 ),
796                 f"Failed to create job {job_id}",
797             )
798
799         def create_compile_job(
800             self,
801             job_id: str,
802             video_id: str,
803             video_path: str,
804             segment_ids: List[int],
805             output_filename: str,
806             locale: Optional[str] = None,
807             owner_id: Optional[str] = None,
808         ) -> bool:
809             """Insert an async REEL-COMPILE job (#329) in state 'queued', `kind`='compile'.
The stitch
810             params (segment_ids / output_filename / locale) ride the JSON `params` column
since they don't
811             map onto the process columns; `video_path` is the target video's source ref (the
table's
812             NOT NULL column) and `video_id` is set up-front (so /api/jobs surfaces it
immediately). The
```

```
813             worker dispatches on `kind` and runs the ffmpeg stitch off the request
thread."""
814             params = json.dumps(
815                 {
816                     "segment_ids": list(segment_ids),
817                     "output_filename": output_filename,
818                     "locale": locale,
819                 }
820             )
821             return self._execute_write(
822                 "INSERT INTO jobs (id, video_path, video_id, status, owner_id, locale, kind,
params) "
823                 "VALUES (?, ?, ?, 'queued', ?, ?, 'compile', ?)",
824                 (job_id, video_path, video_id, owner_id, locale, params),
825                 f"Failed to create compile job {job_id}",
826             )
827
828         def count_jobs_by_locale(self) -> Dict[str, int]:
829             """PMF analytics: how many jobs were submitted per UI locale. Unrecorded locales
(NULL ?
830             CLI / pre-v7 / older clients) bucket under '(unset)'. Returns {locale: count},
busiest first."""
831             with self._connect() as conn:
832                 rows = (
833                     conn.cursor()
834                     .execute(
835                         "SELECT COALESCE(locale, '(unset)') AS loc, COUNT(*) AS n "
836                         "FROM jobs GROUP BY loc ORDER BY n DESC, loc"
837                     )
838                     .fetchall()
839                 )
840             return {str(r[0]): int(r[1]) for r in rows}
841
842         def mark_job_running(self, job_id: str) -> bool:
843             return self._execute_write(
844                 "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
845                 "progress='starting' WHERE id = ?",
846                 (job_id,),
847                 f"Failed to mark job {job_id} running",
848             )
849
850         def set_job_progress(self, job_id: str, progress: str) -> bool:
851             return self._execute_write(
852                 "UPDATE jobs SET progress=? WHERE id = ?",
853                 (progress, job_id),
854                 f"Failed to set progress for job {job_id}",
855             )
856
857         def set_job_video_id(self, job_id: str, video_id: str) -> bool:
858             return self._execute_write(
859                 "UPDATE jobs SET video_id=? WHERE id = ?",
```

17. user (2026-07-02T04:47:30.854Z)

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\training\README.md:16:   needs a feature change, change it THERE and bump
`FEATURE_SCHEMA_VERSION`.
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\infrastructure\database.py:18:SCHEMA_VERSION = 9
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\infrastructure\database.py:107:            # P2d upgrade-safety (no-op for a
fresh DB or one already at SCHEMA_VERSION):
```

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\backend\infrastructure\database.py:110:   if version > SCHEMA_VERSION:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\backend\infrastructure\database.py:113:   f"than this one (knows up to v{SCHEMA_VERSION}). Upgrade the app, or point it at "
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\backend\infrastructure\database.py:119:   if 0 < version < SCHEMA_VERSION:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\backend\features\rally_seq.py:9:MUST bump FEATURE_SCHEMA_VERSION; artifact manifests record the version they were
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\backend\features\rally_seq.py:22:FEATURE_SCHEMA_VERSION = 1
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\backend\features\rally_seq.py:32:   "version": FEATURE_SCHEMA_VERSION,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\docs\CODE_MAP.md:177:[Omitted long matching line]
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\backend\eval\rally_seq_proto.py:49:   FEATURE_SCHEMA_VERSION,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_db_upgrade_safety.py:8:from backend.infrastructure.database import SCHEMA_VERSION, Database, SchemaTooNewError
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_db_upgrade_safety.py:29:   """Ties the SCHEMA_VERSION constant to the migration ladder: a freshly-built DB lands exactly at
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_db_upgrade_safety.py:30:   SCHEMA_VERSION. Adding a _migrate_to_vN without bumping the constant fails here."""
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_db_upgrade_safety.py:33:   assert _user_version(p) == SCHEMA_VERSION
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_db_upgrade_safety.py:45:   Database(p) # ? SCHEMA_VERSION
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_db_upgrade_safety.py:53:   _set_user_version(p, SCHEMA_VERSION + 5) # a DB from a hypothetical newer app
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_db_upgrade_safety.py:67:   assert _user_version(p) == SCHEMA_VERSION # upgraded
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_db_upgrade_safety.py:81:   assert _user_version(dest) == SCHEMA_VERSION
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_import_smoke.py:122:   "assert rs.FEATURE_SCHEMA['version'] == rs.FEATURE_SCHEMA_VERSION\n"
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\docs\PACKAGING_PLAN.md:115:| P2d | **DB upgrade contract** ? `SCHEMA_VERSION` + downgrade guard (`SchemaTooNewError`) + backup-before-migrate (`<db>.bak-v<old>`, WAL-complete) + `backup_database()` export. Byte-identical for fresh/current DBs. *(uninstall manifest deferred to P2b-install.)* | ? | No | ? | [#307](https://github.com/Khelsutra/badminton-

highlight-indexer/pull/307) |
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_training_gen0.py:112:   from backend.features.rally_seq import FEAT_NAMES, FEATURE_SCHEMA_VERSION
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_training_gen0.py:115:   assert payload["feature_schema_version"] == FEATURE_SCHEMA_VERSION

18. user (2026-07-02T04:48:17.909Z)

title: [cost] /api/process re-runs the paid segmenter for an already-processed video ? refresh/"Try again" re-bills Gemini
state: OPEN
author: avidullu (Avi)
labels: bug
comments: 0
assignees:
projects:
milestone:
number: 340
--
Severity: cost + correctness. A page refresh or the SPA's "Try again" re-uploads the same file and POSTs `/api/process` again. Since `video_id` is the **MD5 of the file**, the re-submit carries the **same `video_id`** ? but the engine **re-runs the full (paid) segmenter anyway** instead of returning the existing result. On the Gemini path that's a repeated, real API charge (? $0.10 per 8-min match) for zero new information, plus another multi-minute compute run.

Confirmed current behavior (`backend/main.py`)
`_execute_processing` computes `was_reingest = db_instance.get_video(video_id) is not None` ? i.e. it **detects** the video is already processed ? but uses that only for destroy-after-confirm safety (`_finalize_reingest` prunes/replaces). There is **no "already processed ? reuse, skip the segmenter" path**: it runs the segmenter and replaces the segments regardless. So a re-submit re-bills.

Live trigger (alpha run, 2026-06-22)
An 8.3-min upload finished with 27 rallies, but the SPA's 10-min poll ceiling timed out **~seconds before completion** and showed "Couldn't process ? Try again." Clicking "Try again" would have re-uploaded the identical file and re-run Gemini end-to-end. (We recovered by loading `?video=<id>` instead.) The SPA-side resume is tracked in `avidullu/khelsutra#111`; **this issue is the engine-side guardrail** so we're protected even if a re-submit slips through (defense in depth).

Proposed fix
- Make `/api/process` **idempotent by content hash**: if `video_id` already has valid `play_segments` and no explicit `force`/`reprocess` flag, **return the existing result immediately** (the queryable `video_id`) **without invoking the paid segmenter**.
- Re-processing must be **opt-in** (`force=true` / an explicit "Re-run" action) so it's never accidental.
- Optional earlier short-circuit: if the upload hashes to an existing `video_id`, skip the re-upload too.

Acceptance
- [ ] Re-submitting an already-processed video (same file/hash) returns the cached rallies with **zero** segmenter/Gemini calls (assert no Gemini call in a test).
- [ ] `force=true` still re-runs (explicit), preserving the existing re-ingest destroy-after-confirm safety.
- [ ] Default (no force) is a no-op-cost reuse.
- [ ] Pairs with `avidullu/khelsutra#111` (SPA resume so a refresh doesn't re-trigger in the first place).

Cost context: paid Gemini ? $0.10 per 8-min match; at alpha scale, accidental re-runs are real money + a nasty surprise on a metered key. The M8 cost-ledger exists but doesn't meter

Gemini usage yet, so a silent re-bill wouldn't even show up.
==========
title: [optimization] MD5 segmentation reuse ? skip Gemini re-segmentation when the same video (video_id) is re-uploaded
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:
projects:
milestone:
number: 330
--

Ask (from the owner)

"If I upload the same video, I hope we do an MD5 checksum to reuse the Gemini segmentation."

Today

`video_id = compute_video_id(file)` **is** the MD5 of the file content, and a video's rallies are stored in `play_segments` keyed by `video_id`. But `/api/process` currently **re-runs the full segmentation** (Gemini) on every submit ? re-paying Gemini (paid) and re-waiting (~8 min on the CPU box) even for an already-segmented, byte-identical file. No reuse short-circuit found in `main.py` / `job_worker.py`.

Proposed

On `/api/process`, if `video_id` already has stored segments **for the same segmenter + relevant config**, **reuse** them and skip Gemini (return immediately with the existing rallies). Add an explicit **`force_resegment`** flag to override. The cache key already exists (MD5 `video_id`); this is mostly a short-circuit + the match condition.

Care

- Only reuse when the segmenter and segmentation-affecting config match (don't serve `gemini` segments for a `wasb_hybrid` request).
- Keep the existing "destroy-after-confirm" re-ingest semantics for an explicit re-segment.
- Big win: Gemini is paid + the CPU re-processing is the slow path; reuse makes re-opening a video instant and free.

19. user (2026-07-02T04:48:32.651Z)

```
        Checkpoints the WAL first so ``dest`` is self-contained. Returns ``dest``."""
        parent = os.path.dirname(os.path.abspath(dest))
        if parent:
            os.makedirs(parent, exist_ok=True)
        with self._connect() as conn:
            conn.execute("PRAGMA wal_checkpoint(FULL)")
        shutil.copy2(self.db_path, dest)
        return dest

    def _create_base_tables(self, cursor):
        """Create the videos / play_segments / events base tables (idempotent)."""
        # Videos Table
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS videos (
                id TEXT PRIMARY KEY,
                filepath TEXT NOT NULL,
                processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                status TEXT NOT NULL,
                validation_results TEXT
            )
        """)

        # Play Segments Table (Extensible metadata JSON)
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS play_segments (
```

```
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                video_id TEXT NOT NULL,
                start_time REAL NOT NULL,
                end_time REAL NOT NULL,
                duration REAL NOT NULL,
                server TEXT,
                receiver TEXT,
                metadata TEXT,
                FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
            )
        """)

        # Events Table
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS events (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                segment_id INTEGER NOT NULL,
                timestamp REAL NOT NULL,
                type TEXT NOT NULL,
                player TEXT,
                details TEXT,
                FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
            )
        """)

    def _migrate_to_v1(self, cursor):
        # v1: raw candidate detections (pre-confirmation), detector-agnostic.
        # Common fields are columns; detector-specific metrics go in `stats` JSON,
        # so a new segmenter reuses this table by setting its own `detector`/`stats`.
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS candidate_segments (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                video_id TEXT NOT NULL,
                detector TEXT NOT NULL,
                chunk_index INTEGER,
                start_time REAL NOT NULL,
                end_time REAL NOT NULL,
                duration REAL NOT NULL,
                confidence REAL,
                stats TEXT,
                detection_params TEXT,
                confirmed_segment_id INTEGER,
                human_label TEXT,
                notes TEXT,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
                FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON DELETE SET
NULL
            )
        """)
        cursor.execute("""
            CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                ON candidate_segments(video_id, detector)
        """)
        cursor.execute("PRAGMA user_version = 1")

    def _migrate_to_v2(self, cursor):
```

20. user (2026-07-02T04:48:42.273Z)

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\motion.py:23:    def process_video(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
```

```
process\backend\pipeline\segmenters\motion.py-24-        self,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\motion.py-25-        video_id: str,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\motion.py-26-        video_path: str,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\motion.py-27-        player_pool: Optional[List[str]] =
None,
--
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\motion.py:47:        return Success(value=segments_data)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\motion.py-48-
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\motion.py-49-    def _detect_motion_segments(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\motion.py-50-        self, cap: Any, max_frames:
Optional[int] = None
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\motion.py-51-    ) -> List[Tuple[float, float]]:
--
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\base.py:44:    def process_video(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\base.py-45-        self,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\base.py-46-        video_id: str,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\base.py-47-        video_path: str,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\base.py-48-        player_pool: List[str] = None,
--
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\gemini.py:609:    def process_video(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\gemini.py-610-        self,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\gemini.py-611-        video_id: str,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\gemini.py-612-        video_path: str,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\gemini.py-613-        player_pool: List[str] = None,
--
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\trajectory_hybrid.py:171:    def process_video(  # type:
ignore[override]
```

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\trajectory_hybrid.py-172-        self,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\trajectory_hybrid.py-173-        video_id: str,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\trajectory_hybrid.py-174-        video_path: str,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\trajectory_hybrid.py-175-        player_pool:
Optional[List[str]] = None,
--
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\yolo_hybrid.py:457:    def process_video(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\yolo_hybrid.py-458-        self,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\yolo_hybrid.py-459-        video_id: str,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\yolo_hybrid.py-460-        video_path: str,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\pipeline\segmenters\yolo_hybrid.py-461-        player_pool: List[str] = None,
```

21. user (2026-07-02T04:48:48.509Z)

```
    def process_video(
        self,
        video_id: str,
        video_path: str,
        player_pool: Optional[List[str]] = None,
        max_frames: Optional[int] = None,
    ) -> "Result[List[Dict[str, Any]]]":
        """
        Processes a video to detect play segments using motion heuristics and
        populates SQLite with detected segments. The ``player_pool`` arg is
        accepted for ABC signature compatibility but no longer used (the motion
        segmenter has no ground truth for server/receiver ? see _populate_db).
        """
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            logger.error(f"Segmenter could not open video: {video_path}")
            return Failure(
                message=f"Could not open video: {video_path}", is_recoverable=False
            )

        final_segments = self._detect_motion_segments(cap, max_frames)

        segments_data = self._populate_db(video_id, final_segments)

        return Success(value=segments_data)
```

22. user (2026-07-02T04:48:49.777Z)

```
Found 4 files
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_job_worker_module_seam.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
```

indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_api_cloud_dispatch.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_async_jobs.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_base.py

23. user (2026-07-02T04:48:55.015Z)

```
1       """Async job model (DEFERRED_HARDENING_PLAN #16).
2
3       Contract under test:
4       - POST /api/process ? 202 + job_id (fast-fail 4xx still happens at submit time)
5       - GET /api/jobs/{id} ? {status, progress, result, reports, ...}
6       - job `status` = EXECUTION state (queued|running|done|failed); the pipeline's own
7         verdict (e.g. failed validation) lives in result["status"].
8       - worker crash restores pre-run segments (destroy-AFTER-confirm) and marks the job
failed
9       - startup sweep (fail_stale_jobs) terminalizes jobs orphaned by a restart
10      - migration: fresh DB lands at user_version 2 with a jobs table; v1 DBs upgrade in place
11      """
12
13      import json
14      import sqlite3
15      import time
16      from concurrent.futures import ThreadPoolExecutor
17
18      import pytest
19
20      pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
21      from fastapi.testclient import TestClient
22
23      import backend.main as m
24      from backend.config.models import AppConfig
25      from backend.infrastructure.database import Database
26      from backend.pipeline.validators.base import ValidationResult
27
28
29      class _InlineExecutor:
30          """submit() runs the fn synchronously ? deterministic tests, no threads."""
31
32          def submit(self, fn, *args, **kwargs):
33              fn(*args, **kwargs)
34
35
36      @pytest.fixture
37      def env(tmp_path, monkeypatch):
38          db = Database(str(tmp_path / "t.db"))
39          cfg = AppConfig()
40          monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
41          app = m.create_app(cfg, db)
42          client = TestClient(app)
43          return client, db, tmp_path, monkeypatch
44
45
46      def _video(tmp_path, name="m.mp4"):
47          p = tmp_path / name
48          p.write_bytes(b"not-a-real-video-but-hashable")
49          return p
50
51
52      def _pass():
53          return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
54
```

```
55
56      def _fail():
57          return [
58              ValidationResult(
59                  validator_name="blur", passed=False, message="blurry", details={}
60              )
61          ]
62
63
64      class _FakeSeg:
65          def __init__(self, db, cfg):
66              self.db = db
67
68          def process_video(self, video_id, video_path, *a, **k):
69              sid = self.db.add_play_segment(video_id, 0.0, 5.0)
70              return [
71                  {
72                      "id": sid,
73                      "start_time": 0.0,
74                      "end_time": 5.0,
75                      "duration": 5.0,
76                      "metadata": {},
77                  }
78              ]
79
80
81      class _RaisingSeg:
82          def __init__(self, db, cfg):
83              self.db = db
84
85          def process_video(self, video_id, video_path, *a, **k):
86              self.db.add_play_segment(video_id, 9.0, 10.0)  # half-written row pre-crash
87              raise RuntimeError("boom")
88
89
90      # --- migration ------------------------------------------------------------
91
92
93      def test_migration_fresh_db_is_v3_with_jobs_table(tmp_path):
94          db = Database(str(tmp_path / "fresh.db"))
95          with db._get_connection() as conn:
96              # v9 = async compile (kind/params); v8 = compute-picker; v7 = UI locale; v6 =
cost ledger;
97              # jobs at v2/v3, user_models v5.
98              assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
99              cols = {r["name"] for r in conn.execute("PRAGMA table_info(jobs)").fetchall()}
100         assert {
101             "id",
102             "video_path",
103             "video_id",
104             "status",
105             "progress",
106             "error",
107             "result",
108         } <= cols
109         assert {"policy", "next_poll_at"} <= cols  # v3 self-scheduling columns
110         assert {
111             "locale",
112             "compute",
113             "kind",
114             "params",
115         } <= cols  # v7 locale, v8 compute, v9 compile job
116
117
118     def test_migration_upgrades_v1_db_in_place(tmp_path):
```

```
        path = str(tmp_path / "v1.db")
        conn = sqlite3.connect(path)
        conn.execute(
            "CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT NOT NULL, "
            "processed_at TIMESTAMP, status TEXT NOT NULL, validation_results TEXT)"
        )
        # A genuine v1 DB also has candidate_segments (created in the version<1 block);
include it
        # so the v4 personalization ALTER (which adds user_id to that table) has a table to
alter.
        conn.execute(
            "CREATE TABLE candidate_segments (id INTEGER PRIMARY KEY AUTOINCREMENT, "
            "video_id TEXT NOT NULL, detector TEXT NOT NULL, start_time REAL NOT NULL, "
            "end_time REAL NOT NULL, duration REAL NOT NULL, stats TEXT)"
        )
        conn.execute("PRAGMA user_version = 1")
        conn.commit()
        conn.close()

        db = Database(path)  # ladder should take it 1 ? 9
        with db._get_connection() as c:
            assert c.execute("PRAGMA user_version").fetchone()[0] == 9
            assert c.execute(
                "SELECT name FROM sqlite_master WHERE type='table' AND name='jobs'"
            ).fetchone()
            # Restore v5 personalization coverage (user_models table + user_id on
candidate_segments).
            # These were dropped in prior edit; add (do not replace) per review.
            tables = {
                r[0]
                for r in c.execute(
                    "SELECT name FROM sqlite_master WHERE type='table'"
                ).fetchall()
            }
            assert "user_models" in tables
            cand_cols = {
                r["name"]
                for r in c.execute("PRAGMA table_info(candidate_segments)").fetchall()
            }
            assert "user_id" in cand_cols


    def test_parity_job_model_with_direct_processing(env, tmp_path, monkeypatch):
        """Item 5 positive: direct CLI-style path (m._execute_processing) vs job-worker path
        (submit + _InlineExecutor leading to _run_claimed_job / drain, plus explicit
_run_claimed_job)
        produce equivalent outcomes for the same ProcessRequest shape. Exercises success,
        Failure, and empty-run branches. Real DB queries using vid (no self-compares or
tautologies).
        """
        client, db, tpath, mp = env
        vid_file = _video(tpath)
        vid = m.compute_video_id(str(vid_file))
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)

        req = m.ProcessRequest(video_path=str(vid_file), segmenter="x")

        # Direct CLI-style path
        direct_out = m._execute_processing(m.global_config, m.db_instance, req)
        segs = db.query_play_segments(vid, {})
        assert len(segs) >= 1, f"direct path must affect segments for vid {vid}"
        assert direct_out.get("result", {}).get("status") in (
            None,
            "success",
```

```
            "processed",
        ) or "play_segments" in str(direct_out)

        # Job-worker path via public API submit (exercises job creation + inline executor +
internal _run_claimed_job)
        rj = client.post(
            "/api/process", json={"video_path": str(vid_file), "segmenter": "x"}
        )
        job_out = client.get(f"/api/jobs/{rj.json()['job_id']}").json()
        segs_j = db.query_play_segments(vid, {})
        assert job_out["result"]["status"] == "success"
        assert len(segs_j) >= 1

        # Explicit job-worker via _run_claimed_job (create + mark running + drive the
claimed runner)
        vid_file2 = _video(tpath, name="m2.mp4")
        vid2 = m.compute_video_id(str(vid_file2))
        jid = f"parity-{vid2[:8]}"
        db.create_job(jid, str(vid_file2))
        db.mark_job_running(jid)
        job_row = db.get_job(jid)
        m._run_claimed_job(job_row, m.global_config, m.db_instance)
        segs2 = db.query_play_segments(vid2, {})
        assert len(segs2) >= 1, (
            f"_run_claimed_job path must produce segments for vid {vid2}"
        )

        # Failure branch (direct + job submit)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
        req_f = m.ProcessRequest(video_path=str(vid_file))
        direct_f = m._execute_processing(m.global_config, m.db_instance, req_f)
        rf = client.post("/api/process", json={"video_path": str(vid_file)})
        jobf = client.get(f"/api/jobs/{rf.json()['job_id']}").json()
        assert (
            direct_f.get("result", {}).get("status") == "failed"
            or jobf["result"]["status"] == "failed"
        )

        # Empty-run branch (success verdict but zero play_segments from this execution)
        class _EmptySeg:
            def __init__(self, db, cfg):
                self.db = db

            def process_video(self, video_id, video_path, *a, **k):
                return []

        mp.setattr(m.segmenter_registry, "get", lambda name: _EmptySeg)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        req_e = m.ProcessRequest(video_path=str(vid_file))
        de = m._execute_processing(m.global_config, m.db_instance, req_e)
        re = client.post("/api/process", json={"video_path": str(vid_file)})
        je = client.get(f"/api/jobs/{re.json()['job_id']}").json()
        assert de.get("result", {}).get("status") in (None, "success")
        assert je["result"]["status"] == "success"
        # (reingest safety for prior on empty/fail covered in dedicated tests)


    # --- submit-time fast-fail ------------------------------------------------------


    def test_submit_returns_202_with_job_id(env):
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
```

```
        r = client.post(
            "/api/process", json={"video_path": str(vid_file), "segmenter": "x"}
        )
        assert r.status_code == 202
        body = r.json()
        assert body["job_id"]
        assert body["status"] == "queued"
        assert body["poll"].endswith(body["job_id"])


    def test_submit_rejects_null_byte_path(env):
        client = env[0]
        r = client.post("/api/process", json={"video_path": "a\x00b.mp4"})
        assert r.status_code == 400


    def test_submit_missing_file_is_404(env):
        client, db, tmp_path, mp = env
        r = client.post("/api/process", json={"video_path": str(tmp_path / "nope.mp4")})
        assert r.status_code == 404


    def test_submit_unknown_segmenter_400_and_no_job_row(env):
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        mp.setattr(m.segmenter_registry, "get", lambda name: None)
        mp.setattr(m.segmenter_registry, "list_available", lambda: {"motion": "..."})
        r = client.post(
            "/api/process", json={"video_path": str(vid_file), "segmenter": "nope"}
        )
        assert r.status_code == 400
        assert "Available segmenters" in r.json()["detail"]
        with db._get_connection() as c:
            assert c.execute("SELECT COUNT(*) FROM jobs").fetchone()[0] == 0


    # --- end-to-end job lifecycle (inline executor = deterministic) ---------------------


    def test_job_done_with_segments_and_video_id(env):
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)

        job_id = client.post(
            "/api/process", json={"video_path": str(vid_file), "segmenter": "x"}
        ).json()["job_id"]
        r = client.get(f"/api/jobs/{job_id}")
        assert r.status_code == 200
        body = r.json()
        assert body["status"] == "done"
        assert body["progress"] == "finished"
        assert body["error"] is None
        assert body["result"]["status"] == "success"
        assert len(body["result"]["play_segments"]) == 1
        assert "reports" in body  # None without obsreport; key must exist per the contract

        vid = m.compute_video_id(str(vid_file))
        assert body["video_id"] == vid
        assert len(db.query_play_segments(vid, {})) == 1
        assert db.get_video(vid)["status"] == "processed"


    def test_job_done_but_pipeline_verdict_failed_preserves_prior(env):
```

```
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        vid = m.compute_video_id(str(vid_file))
        db.add_video(vid, str(vid_file), "processed", [])
        db.add_play_segment(vid, 1.0, 2.0)  # prior good result
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))

        job_id = client.post("/api/process", json={"video_path": str(vid_file)}).json()[
            "job_id"
        ]
        body = client.get(f"/api/jobs/{job_id}").json()
        # Worker finished fine ? the PIPELINE verdict is the failure.
        assert body["status"] == "done"
        assert body["result"]["status"] == "failed"
        # PR-C semantics survive the async move: prior segments intact.
        assert len(db.query_play_segments(vid, {})) == 1


    def test_worker_crash_marks_failed_and_restores_prior_segments(env):
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        vid = m.compute_video_id(str(vid_file))
        db.add_video(vid, str(vid_file), "processed", [])
        db.add_play_segment(vid, 1.0, 2.0)  # prior good result
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _RaisingSeg)

        job_id = client.post(
            "/api/process", json={"video_path": str(vid_file), "segmenter": "x"}
        ).json()["job_id"]
        body = client.get(f"/api/jobs/{job_id}").json()
        assert body["status"] == "failed"
        assert "RuntimeError" in body["error"]
        assert body["result"] is None
        # The half-written 9.0?10.0 row was pruned; the prior 1.0?2.0 row survives.
        segs = db.query_play_segments(vid, {})
        assert len(segs) == 1 and segs[0]["start_time"] == 1.0


    def test_progress_stages_are_recorded(env):
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)

        stages = []
        real = db.set_job_progress

        def spy(job_id, progress):
            stages.append(progress)
            return real(job_id, progress)

        mp.setattr(db, "set_job_progress", spy)
        client.post("/api/process", json={"video_path": str(vid_file), "segmenter": "x"})
        assert "hashing" in stages
        assert any(s.startswith("segmenting") for s in stages)


    # --- polling endpoint ------------------------------------------------------------


    def test_get_unknown_job_is_404(env):
        client = env[0]
        assert client.get("/api/jobs/deadbeef").status_code == 404

```

```
372
373     def test_create_compile_job_persists_kind_and_params(tmp_path):
374         """#329: an async compile job rides the jobs table with kind='compile' + a JSON
params payload
375         (segment_ids/output_filename/locale); a process job leaves kind NULL (? process ?
byte-identical)."""
376         db = Database(str(tmp_path / "jobs.db"))
377         assert db.create_compile_job(
378             "cj", "vidX", "gs://b/x.mp4", [1, 2, 3], "reel.mp4", locale="kn"
379         )
380         job = db.get_job("cj")
381         assert job["kind"] == "compile"
382         assert job["video_id"] == "vidX"
383         params = json.loads(job["params"])
384         assert params["segment_ids"] == [1, 2, 3]
385         assert params["output_filename"] == "reel.mp4"
386         assert params["locale"] == "kn"
387         db.create_job("pj", "/v.mp4")  # a process job
388         assert db.get_job("pj")["kind"] is None
389
390
391     def test_worker_dispatches_compile_kind_to_execute_compile(env):
392         """The claimed-job dispatcher routes kind='compile' to _execute_compile (NOT
_execute_processing),
393         records the result as 'done', and skips the process-only cost/flywheel hooks."""
394         _client, db, _tmp_path, mp = env
395         seen = {}
396
397         def _fake_compile(cfg, db, job, progress=None):
398             seen["id"] = job["id"]
399             return {
400                 "status": "success",
401                 "video_id": "vidX",
402                 "download_url": "/api/highlights/vidX/r.mp4",
403             }
404
405         mp.setattr(m, "_execute_compile", _fake_compile)
406         mp.setattr(
407             m,
408             "_execute_processing",
409             lambda *a, **k: pytest.fail("a compile job must not hit _execute_processing"),
410         )
411         db.create_compile_job("cj2", "vidX", "/v.mp4", [1], "r.mp4")
412         m._run_claimed_job(db.get_job("cj2"), m.global_config, m.db_instance)
413         assert seen["id"] == "cj2"
414         done = db.get_job("cj2")
415         assert done["status"] == "done"
416         assert done["result"]["download_url"] == "/api/highlights/vidX/r.mp4"
417
418
419     def test_create_job_persists_and_reconstructs_compute(tmp_path):
420         """v8: the compute-picker choice must round-trip create_job ? get_job ?
_job_to_request so the
421         worker honours it across the async submit?reconstruct boundary. NULL ? server
default (the
422         pre-picker default-OFF behaviour). Regression guard for the dropped-across-boundary
bug."""
423         db = Database(str(tmp_path / "jobs.db"))
424         db.create_job("j-cloud", "gs://b/clip.mp4", compute="cloud")
425         db.create_job("j-default", "/v.mp4")  # no compute ? NULL ? server default
426         assert db.get_job("j-cloud")["compute"] == "cloud"
427         assert db.get_job("j-default")["compute"] is None
428         assert m._job_to_request(db.get_job("j-cloud")).compute == "cloud"
429         assert m._job_to_request(db.get_job("j-default")).compute is None
430
```

```
431
432     # --- crash recovery ----------------------------------------------------------
433
434
435     def test_fail_stale_jobs_terminalizes_queued_and_running_only(env):
436         client, db, tmp_path, mp = env
437         db.create_job("q1", "/v.mp4")
438         db.create_job("r1", "/v.mp4")
439         db.mark_job_running("r1")
440         db.create_job("d1", "/v.mp4")
441         db.mark_job_done("d1", {"status": "success"})
442
443         swept = db.fail_stale_jobs("interrupted by server restart")
444         assert swept == 2
445         assert db.get_job("q1")["status"] == "failed"
446         assert db.get_job("r1")["status"] == "failed"
447         assert "restart" in db.get_job("q1")["error"]
448         assert db.get_job("d1")["status"] == "done"  # terminal states untouched
449
450
451     # --- real executor (cross-thread SQLite/WAL sanity) --------------------------------
452
453
454     def test_real_executor_completes_job(env):
455         client, db, tmp_path, mp = env
456         vid_file = _video(tmp_path)
457         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
458         mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
459         pool = ThreadPoolExecutor(max_workers=1)
460         mp.setattr(m, "_get_executor", lambda: pool)
461         try:
462             job_id = client.post(
463                 "/api/process", json={"video_path": str(vid_file), "segmenter": "x"}
464             ).json()["job_id"]
465             deadline = time.time() + 10.0
466             body = None
467             while time.time() < deadline:
468                 body = client.get(f"/api/jobs/{job_id}").json()
469                 if body["status"] in ("done", "failed"):
470                     break
471                 time.sleep(0.05)
472             assert body is not None and body["status"] == "done"
473             assert body["result"]["status"] == "success"
474         finally:
475             pool.shutdown(wait=True)
476
```

24. assistant (2026-07-02T04:49:48.053Z)

Now let me check where ProcessRequest is constructed and how the CLI path works, plus look at the validators cost consideration.

25. user (2026-07-02T04:49:50.850Z)

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\api\job_worker.py:77:    return _main.ProcessRequest(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\backend\main.py:313:class ProcessRequest(BaseModel):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_async_jobs.py:170:    req = m.ProcessRequest(video_path=str(vid_file),
segmenter="x")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
```

```
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_async_jobs.py:206:    req_f = m.ProcessRequest(video_path=str(vid_file))
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_async_jobs.py:225:    req_e = m.ProcessRequest(video_path=str(vid_file))
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py:147:
m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py:166:
m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py:192:
m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py:205:
m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py:224:        m.ProcessRequest(video_path=local_path,
segmenter="wasb_hybrid")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py:265:        m.ProcessRequest(video_path=str(video),
segmenter="motion")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py:294:
m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py:308:
m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py:351:        m.ProcessRequest(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py:394:        m.ProcessRequest(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py:425:        m.ProcessRequest(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py:443:            m.ProcessRequest(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py:598:        m.ProcessRequest(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py:620:        m.ProcessRequest(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py:644:        m.ProcessRequest(video_path=str(video),
segmenter="motion")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py:672:        m.ProcessRequest(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py:699:    out =
```

```
m._execute_processing(m.ProcessRequest(video_path=str(video)))
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_locale_analytics.py:93:    assert ProcessRequest(video_path="x", locale="hi-
IN").locale == "hi-IN"
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_locale_analytics.py:94:    assert ProcessRequest(video_path="x").locale is
None
```

26. user (2026-07-02T04:49:59.433Z)

```
196-
197-def test_cloud_failure_marks_failed_no_rows(cloud_env):
198-    db, _bucket, monkeypatch = cloud_env
199-    # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-
confirm).
200:    db.add_video("vidLocalPrior", "gs://b/clip.mp4", "processed", [])
201-    fake = _FakeCloudClient(returncode=1)
202-    _inject_fake(monkeypatch, fake)
203-
204-    out = m._execute_processing(
--
248-        def __init__(self, db_, cfg_):
249-            self.db = db_
250-
251-        def process_video(self, vid, path, *a, **k):
252:            sid = self.db.add_play_segment(vid, 0.0, 5.0)
253-            return [
254-                {
255-                    "id": sid,
256-                    "start_time": 0.0,
--
321-    def __init__(self, db_, cfg_):
322-        self.db = db_
323-
324-    def process_video(self, vid, path, *a, **k):
325:        sid = self.db.add_play_segment(vid, 0.0, 5.0)
326-        return [
327-            {
328-                "id": sid,
329-                "start_time": 0.0,
```

27. user (2026-07-02T04:50:01.149Z)

```
1832:def main():
1836:    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
1916:        if not args.video_path:
1917:            print("[ERROR] Please provide --video-path to extract a segment.")
1935:            f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s
from {args.video_path}..."
1950:                args.video_path, [(args.trim_start, args.trim_end)], out_name
1958:        if not args.video_path:
1959:            print("[ERROR] Please provide --video-path to debug a specific clip.")
1961:        run_cli_diagnose_clip(args.video_path, args.debug_clip)
2042:    if args.video_path and not args.server and not args.app:
2043:        print(f"[CLI] Processing video file: {args.video_path}")
2049:            input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
2054:            print(f"[FAIL] Video file not found: {args.video_path}")
2056:        local_video = storage.acquire_local_input(args.video_path, storage_cfg)
2071:            video_id, args.video_path, status, [r.model_dump() for r in val_results]
2142:    if args.app or args.server or not args.video_path:
```

28. user (2026-07-02T04:50:12.547Z)

```
"""Re-ingesting a video must not destroy its prior good segments.

Regression for the destroy-before-confirm finding: add_video used INSERT OR REPLACE,
which cascade-deleted play/candidate segments on any status update (including a failed
re-validation or the 'processed' mark written before the segmenter even ran).
"""

from backend.infrastructure.database import Database


def _seed(db, vid="vid1"):
    db.add_video(vid, f"{vid}.mp4", "processed", [])
    sid = db.add_play_segment(vid, 1.0, 4.0)
    cid = db.add_candidate_segment(vid, "wasb_hybrid", 1.0, 4.0)
    return sid, cid


def test_add_video_upsert_preserves_segments(tmp_path):
    db = Database(str(tmp_path / "t.db"))
    _seed(db)
    # A failed re-validation only updates status ? it must NOT wipe prior segments.
    db.add_video("vid1", "vid1.mp4", "failed", [{"validator": "blur", "passed": False}])
    assert len(db.query_play_segments("vid1", {})) == 1
    assert len(db.query_candidate_segments("vid1")) == 1
    assert db.get_video("vid1")["status"] == "failed"


def test_prune_keeps_new_drops_old(tmp_path):
    db = Database(str(tmp_path / "t.db"))
    db.add_video("vid1", "vid1.mp4", "processed", [])
    db.add_play_segment("vid1", 1.0, 2.0)  # old
    db.add_candidate_segment("vid1", "wasb_hybrid", 1.0, 2.0)
    play_wm, cand_wm = db.segment_watermarks("vid1")
    db.add_play_segment("vid1", 10.0, 12.0)  # new run
    db.add_candidate_segment("vid1", "wasb_hybrid", 10.0, 12.0)
    # Successful run: keep new, drop old.
    db.prune_segments("vid1", play_upto=play_wm, cand_upto=cand_wm)
    segs = db.query_play_segments("vid1", {})
    assert len(segs) == 1 and segs[0]["start_time"] == 10.0
    assert len(db.query_candidate_segments("vid1")) == 1


def test_prune_keeps_old_drops_new_on_empty_run(tmp_path):
    db = Database(str(tmp_path / "t.db"))
    db.add_video("vid1", "vid1.mp4", "processed", [])
    db.add_play_segment("vid1", 1.0, 2.0)  # prior good result
    db.add_candidate_segment("vid1", "wasb_hybrid", 1.0, 2.0)
    play_wm, cand_wm = db.segment_watermarks("vid1")
    # Simulate a re-run that wrote partial candidates then produced 0 final segments.
    db.add_candidate_segment("vid1", "wasb_hybrid", 30.0, 31.0)
    # Empty run: drop the new partial rows, keep the prior good ones.
    db.prune_segments("vid1", play_after=play_wm, cand_after=cand_wm)
    segs = db.query_play_segments("vid1", {})
    assert len(segs) == 1 and segs[0]["start_time"] == 1.0
    cands = db.query_candidate_segments("vid1")
    assert len(cands) == 1 and cands[0]["start_time"] == 1.0


def test_watermarks_zero_when_empty(tmp_path):
    db = Database(str(tmp_path / "t.db"))
    db.add_video("vid1", "vid1.mp4", "processed", [])
    assert db.segment_watermarks("vid1") == (0, 0)


def test_reingest_safety_parity_with_duplication_paths(tmp_path):
```

```
    """Item 5 positive: reingest safety (preserve prior good on failure/empty) holds in
    duplication scenarios (CLI-style vs job worker paths).
    """
    db = Database(str(tmp_path / "t.db"))
    _seed(db)
    # failure path preserves
    db.add_video("vid1", "vid1.mp4", "failed", [{"validator": "blur", "passed": False}])
    assert len(db.query_play_segments("vid1", {})) == 1
    assert len(db.query_candidate_segments("vid1")) == 1
    # empty path (re-run with 0 final) drops new, keeps prior
    play_wm, cand_wm = db.segment_watermarks("vid1")
    db.add_candidate_segment("vid1", "wasb_hybrid", 30.0, 31.0)
    db.prune_segments("vid1", play_after=play_wm, cand_after=cand_wm)
    segs = db.query_play_segments("vid1", {})
    assert len(segs) == 1 and segs[0]["start_time"] == 1.0
    cands = db.query_candidate_segments("vid1")
    assert len(cands) == 1 and cands[0]["start_time"] == 1.0
```

29. user (2026-07-02T04:50:28.062Z)

```
<persisted-output>
Output too large (28.5KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-
Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\tool-
results\toolu_01KdfAnk1ouYJto7hRh7Mb7N.txt

Preview (first 2KB):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_annotations.py:33:    db.add_video("abc123", str(vid), "processed", [])
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_annotations.py-34-
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_annotations.py-35-    r = client.get("/api/source/abc123")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_annotations.py-36-    assert r.status_code == 200
--
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_annotations.py:48:    db.add_video("gone", str(tmp_path / "not-
there.mp4"), "processed", [])
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_annotations.py-49-    assert client.get("/api/source/gone").status_code
== 404
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_annotations.py-50-
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_annotations.py-51-
--
...
</persisted-output>
```

30. user (2026-07-02T04:50:38.143Z)

```
Found 7 files
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_server_entrypoint.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
```

```
process\tests\test_locale_analytics.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_job_worker_loop.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_edge_auth.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_cloud_dispatch.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_api_endpoints.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_async_jobs.py
```

31. user (2026-07-02T04:50:49.901Z)

```
=== tests/test_server_entrypoint.py
=== tests/test_locale_analytics.py
=== tests/test_job_worker_loop.py
71:        sid = self.db.add_play_segment(video_id, 0.0, 5.0)
152:            sid = self.db.add_play_segment(video_id, 0.0, 5.0)
=== tests/test_edge_auth.py
=== tests/test_api_endpoints.py
61:        sid = self.db.add_play_segment(video_id, 0.0, 5.0)
147:    db.add_video("vid_a", "/a.mp4", "processed", [])
148:    db.add_video("vid_b", "/b.mp4", "processed", [])
158:    db.add_video("md5abc", "/srv/uploads/Saturday match_2026.mp4", "processed", [])
159:    db.add_video("md5gcs", "gs://bucket/videos/Sunday_final.MP4", "processed", [])
172:    db.add_video(
179:    db.add_video("vid_bare", "/u/bare.mp4", "processed", [])  # no probe details
265:    db.add_video(vid, str(vid_file), "processed", [])
266:    db.add_play_segment(vid, 1.0, 2.0)  # prior good result
377:    db.add_video("vid", "/x.mp4", "processed", [])
378:    db.add_play_segment("vid", 1.0, 4.0)
386:    db.add_video("vid", str(_video(tmp_path)), "processed", [])
387:    sid = db.add_play_segment("vid", 1.0, 4.0)
401:    db.add_video("vid", str(_video(tmp_path)), "processed", [])
402:    sid = db.add_play_segment("vid", 1.0, 4.0)
433:    db.add_video(
436:    sid = db.add_play_segment("bvid", 1.0, 4.0)  # no shot_count ? exempt from the floor
483:    db.add_video("vid", str(_video(tmp_path)), "processed", [])
484:    sid = db.add_play_segment("vid", 1.0, 4.0)
508:    db.add_video("vid", str(_video(tmp_path)), "processed", [])
=== tests/test_job_worker_module_seam.py
=== tests/test_base.py
```

32. user (2026-07-02T04:50:56.524Z)

```
        ).status_code
        == 202
    )
    assert db.count_jobs_by_locale() == {"kn": 1}


def test_process_validation_failure_preserves_prior_segments(env):
    client, db, tmp_path, mp = env
    vid_file = _video(tmp_path)
    vid = m.compute_video_id(str(vid_file))
    db.add_video(vid, str(vid_file), "processed", [])
    db.add_play_segment(vid, 1.0, 2.0)  # prior good result
    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
    r = client.post("/api/process", json={"video_path": str(vid_file)})
```

```
    assert r.status_code == 202
    job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
    assert job["status"] == "done"  # worker finished; the PIPELINE verdict failed
    assert job["result"]["status"] == "failed"
    # PR-C: a failed re-validation must NOT cascade-delete prior segments.
    assert len(db.query_play_segments(vid, {})) == 1


def test_process_rejects_null_byte_path(env):
    client = env[0]
    r = client.post("/api/process", json={"video_path": "a\x00b.mp4"})
    assert r.status_code == 400
```

==
Item 4 positive tests (output layout honors configured output_dir)
==

```
def test_process_uses_configured_output_dir_not_literal_output(
    env, tmp_path, monkeypatch
):
    """Positive test for Item 4: when output.output_dir overridden, artifacts/paths
    use the configured root; no leakage to literal 'output/' for the video_id.

    This would have caught re-introduction of hard-coded os.path.join("output", ...)
    (see main.py:548/619 and similar in compiler/detectors). Uses the existing
    env fixture + tmp_path for isolation, matching reingest safety + compiler patterns.
    """
    client, db, tpath, mp = env
    vid_file = _video(tpath)
    vid = m.compute_video_id(str(vid_file))
```

33. user (2026-07-02T04:51:15.089Z)

```
"""P3 worker self-scheduling loop (EXECUTION_POLICY.md P3).

The P2 slice added the DB primitives (`claim_due_job`, `set_job_waiting`,
`seconds_until_next_due`); this slice wires the `main.py` worker to them so the worker
DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job that raises
`JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and re-claimed when
due ? instead of the old one-shot `submit(_run_job, ...)`.

Covered:
- `_drain_due_jobs` claims + runs queued jobs to a terminal state (no regression on the
  existing sync/submit path ? same `done`/`failed`/verdict semantics);
- earliest-due ordering across queued + due-waiting;
- a `JobWaiting` parks the job (no terminal state) and a later drain (once due) resumes it;
- not-yet-due parked jobs are left alone and a wake-timer is armed for them;
- the wake-scheduling decision (`_schedule_next_drain_if_waiting`) is asserted with the
  timer seam stubbed ? no real waiting, no leaked threads.

The real-timer seam (`_arm_wake_timer`) is monkeypatched to a recorder in every test that
can park a job, so nothing actually sleeps and no daemon timer outlives the test.
"""

import sqlite3

import pytest

pytest.importorskip("httpx")
from fastapi.testclient import TestClient
```

```
import backend.main as m
from backend.config.models import AppConfig
from backend.infrastructure.database import Database
from backend.pipeline.validators.base import ValidationResult


class _InlineExecutor:
    """submit() runs fn synchronously ? deterministic, single-threaded tests."""

    def submit(self, fn, *args, **kwargs):
        fn(*args, **kwargs)


@pytest.fixture
def env(tmp_path, monkeypatch):
    db = Database(str(tmp_path / "t.db"))
    cfg = AppConfig()
    monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
    # Stub the only real-timer seam so a parked job never arms a live daemon timer; tests
    # that care about scheduling assert against this recorder instead.
    armed: list = []
    monkeypatch.setattr(m, "_arm_wake_timer", lambda delay, *a: armed.append(delay))
    app = m.create_app(cfg, db)
    client = TestClient(app)
    return client, db, tmp_path, monkeypatch, armed


def _video(tmp_path, name="m.mp4"):
    p = tmp_path / name
    p.write_bytes(b"not-a-real-video-but-hashable")
    return p


def _pass():
    return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]


class _FakeSeg:
    def __init__(self, db, cfg):
        self.db = db

    def process_video(self, video_id, video_path, *a, **k):
        sid = self.db.add_play_segment(video_id, 0.0, 5.0)
        return [
            {
                "id": sid,
                "start_time": 0.0,
                "end_time": 5.0,
                "duration": 5.0,
                "metadata": {},
            }
        ]


def _wire_happy(mp, client_env_video):
    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
    mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
```

--- no-regression: the drain runs a queued job exactly like the old submit path ---------

```
def test_drain_runs_queued_job_to_done(env):
    client, db, tmp_path, mp, _armed = env
```

```
    vid_file = _video(tmp_path)
    _wire_happy(mp, vid_file)

    # Submit via the public endpoint (which now submits a drain tick, not _run_job).
    job_id = client.post(
        "/api/process", json={"video_path": str(vid_file), "segmenter": "x"}
    ).json()["job_id"]
    body = client.get(f"/api/jobs/{job_id}").json()
    assert body["status"] == "done"
    assert body["result"]["status"] == "success"
    assert len(body["result"]["play_segments"]) == 1
    vid = m.compute_video_id(str(vid_file))
    assert body["video_id"] == vid
    assert len(db.query_play_segments(vid, {})) == 1


def test_drain_returns_count_and_drains_all_queued(env):
    client, db, tmp_path, mp, _armed = env
    _wire_happy(mp, None)
    # Two queued rows created directly (no endpoint), distinct video_paths.
    a, b = _video(tmp_path, "a.mp4"), _video(tmp_path, "b.mp4")
    db.create_job("a", str(a), segmenter="x")
    db.create_job("b", str(b), segmenter="x")

    ran = m._drain_due_jobs(m.global_config, m.db_instance)
    assert ran == 2
    assert db.get_job("a")["status"] == "done"
    assert db.get_job("b")["status"] == "done"
```

34. user (2026-07-02T04:51:41.564Z)

```
"""API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).

Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest the rally
intervals back into the API DB, so /api/query + /api/compile (local stitch) are unchanged ? all
GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json + intervals.csv + the
done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
compute_target=cloud_run
dispatches; every other target runs the in-process segmenter byte-identically.
"""

import csv
import json
import os

import pytest

pytest.importorskip("httpx")
from fastapi.testclient import TestClient

import backend.compute as compute_pkg
import backend.main as m
from backend.compute.cloud_run import CloudRunJobClient
from backend.config.models import AppConfig
from backend.infrastructure.database import Database
from backend.infrastructure.execution_policy import ExecutionPolicy
from backend.pipeline.validators.base import ValidationResult

_FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)


def _pass():
    return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
```

```
class _InlineExecutor:
    def submit(self, fn, *a, **k):
        fn(*a, **k)


class _FakeCloudClient(CloudRunJobClient):
    """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv + the
done-marker
    into the local bucket exactly where the real worker would, so the bucket-poll + ingest
resolve."""

    def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
        self.calls: list[list[str]] = []
        self.video_id, self.returncode = video_id, returncode
        self.rows = (
            rows
            if rows is not None
            else [(2.74, 6.91, 5, "winner"), (8.74, 11.38, 3, "unforced_error")]
        )

    def run_job(self, job_name, argv, *, region, project=None):
        self.calls.append(list(argv))
        out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
        done_uri = argv[argv.index("--done-uri") + 1]
        if self.returncode == 0:
            outputs = os.path.join(out_uri, "outputs", self.video_id)
            os.makedirs(outputs, exist_ok=True)
            with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
                json.dump(
                    {
                        "video_id": self.video_id,
                        "segment_count": len(self.rows),
                        "status": "success",
                    },
                    f,
                )
            with open(
                os.path.join(outputs, "intervals.csv"),
                "w",
                newline="",
                encoding="utf-8",
            ) as f:
                w = csv.writer(f)
                w.writerow(["start", "end", "shot_count", "ending_reason"])
                for s, e, sc, er in self.rows:
                    w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
        os.makedirs(os.path.dirname(done_uri), exist_ok=True)
        with open(done_uri, "w", encoding="utf-8") as f:
            json.dump(
                {
                    "video_id": self.video_id,
                    "returncode": self.returncode,
                    "segment_count": len(self.rows),
                    "status": "success",
                },
                f,
            )
        return "exec-1"


@pytest.fixture
def cloud_env(tmp_path, monkeypatch):
    """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the bucket
read an
    identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
```

```
hash+validate."""
    db = Database(str(tmp_path / "t.db"))  # noqa: F841
    monkeypatch.setattr(m, "db_instance", db)
    monkeypatch.setattr(m, "db_instance", db)
    bucket = tmp_path / "bucket"
    bucket.mkdir()
    cfg = AppConfig.model_validate(
        {  # noqa: F841
            "deployment": {"compute_target": "cloud_run"},
            "storage": {"backend": "local", "output_root": str(bucket)},
        }
    )
    monkeypatch.setattr(m, "global_config", cfg)
    monkeypatch.setattr(m, "global_config", cfg)
    monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
    # A real gs:// input would be fetched to hash+validate; return a local fake so that path is
offline.
    fake_local = tmp_path / "fetched.mp4"
    fake_local.write_bytes(b"FAKEVIDEOBYTES")
    monkeypatch.setattr(
        m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
    )
    monkeypatch.setattr(
        m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
    )
    return db, bucket, monkeypatch
```

35. user (2026-07-02T04:51:50.736Z)

```
        lambda config, **kw: real(
            config, run_client=fake, policy=_FAST, token="tok", **kw
        ),
    )


def _meta(row):
    md = row.get("metadata")
    return json.loads(md) if isinstance(md, str) else (md or {})


def test_cloud_dispatch_ingests_segments(cloud_env):
    db, _bucket, monkeypatch = cloud_env
    fake = _FakeCloudClient()
    _inject_fake(monkeypatch, fake)

    out = m._execute_processing(
        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
    )

    assert out["status"] == "success"
    assert len(out["play_segments"]) == 2
    rows = db.query_play_segments(out["video_id"], {})
    assert len(rows) == 2
    metas = [_meta(r) for r in rows]
    assert all(md["source"] == "cloud_run" for md in metas)
    assert {md.get("shot_count") for md in metas} == {5, 3}
    assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}


def test_cloud_dispatch_builds_correct_spec(cloud_env):
    _db, bucket, monkeypatch = cloud_env
    fake = _FakeCloudClient()
    _inject_fake(monkeypatch, fake)

    m._execute_processing(
```

```
        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
    )

    assert len(fake.calls) == 1  # exactly one Cloud Run Job execution
    argv = fake.calls[0]
    assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
    assert argv[argv.index("--out-uri") + 1] == str(bucket)
    assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
    joined = " ".join(argv)
    assert (
        "deployment.compute_target=local_gpu" in joined
    )  # container runs INLINE (never re-dispatch)
    assert "indexing.detector_impl=native" in joined
    assert (
        "indexing.skip_ai_handoff=false" in joined
    )  # the engine's AI-handoff stance is forwarded


def test_cloud_dispatch_forwards_skip_ai_handoff_true(cloud_env):
    """When the engine runs no-AI (skip_ai_handoff=True), the cloud job must too ? else it tries
the
    Gemini handoff (which the cloud job has no key for) and yields 0 rallies. The flag is
forwarded."""
    _db, _bucket, monkeypatch = cloud_env
    m.global_config.indexing.skip_ai_handoff = True
    fake = _FakeCloudClient()
    _inject_fake(monkeypatch, fake)
    m._execute_processing(
        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
    )
    assert "indexing.skip_ai_handoff=true" in " ".join(fake.calls[0])


def test_cloud_failure_marks_failed_no_rows(cloud_env):
    db, _bucket, monkeypatch = cloud_env
    # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-confirm).
    db.add_video("vidLocalPrior", "gs://b/clip.mp4", "processed", [])
    fake = _FakeCloudClient(returncode=1)
    _inject_fake(monkeypatch, fake)

    out = m._execute_processing(
        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
    )
    assert out["status"] == "failed"
    assert "returncode 1" in out["message"]
    assert out["play_segments"] == []
    assert db.query_play_segments(out["video_id"], {}) == []
    # It DID dispatch + run remotely (then failed rc=1), so the path is honestly cloud_run +
dispatched.
    assert (
        out["compute"]["path"] == "cloud_run" and out["compute"]["dispatched"] is True
    )


def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
    _db, _bucket, monkeypatch = cloud_env
    fake = _FakeCloudClient()
    _inject_fake(monkeypatch, fake)
    local_path = str(tmp_path / "local_only.mp4")

    out = m._execute_processing(
        m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
    )
    assert out["status"] == "failed"
    assert "cloud input" in out["message"]
```

```
    assert fake.calls == []  # never dispatched a local-only path


def test_default_off_runs_in_process(tmp_path, monkeypatch):
    """compute_target='' ? get_compute_backend returns None ? the in-process segmenter runs
    byte-identically (the gate is inert). No Cloud Run dispatch."""
    db = Database(str(tmp_path / "t.db"))  # noqa: F841
    monkeypatch.setattr(m, "db_instance", db)
    monkeypatch.setattr(m, "db_instance", db)
    cfg = AppConfig()  # compute_target defaults to ""  # noqa: F841
    monkeypatch.setattr(m, "global_config", cfg)
    monkeypatch.setattr(m, "global_config", cfg)
    monkeypatch.setattr(m, "global_config", cfg)
```

36. user (2026-07-02T04:51:58.948Z)

```
                "end_time": 5.0,
                "duration": 5.0,
                "metadata": {},
            }
        ]
```

--- item 3: SPA compute-picker (per-request compute override + cloud_available capab

```
def test_per_request_local_forces_in_process(cloud_env):
    """Server configured for cloud (compute_target=cloud_run), but a per-request compute='local'
    forces the in-process segmenter and NEVER dispatches to Cloud Run."""
    db, _bucket, monkeypatch = cloud_env
    fake = _FakeCloudClient()
    _inject_fake(monkeypatch, fake)
    monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)

    out = m._execute_processing(
        m.global_config,
        m.db_instance,
        m.ProcessRequest(
            video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
        ),
    )
    assert out["status"] == "success"
    assert len(out["play_segments"]) == 1
    assert "cloud_run" not in out["message"]
    assert (
        fake.calls == []
    )  # explicit "run on my machine" ? no dispatch despite a cloud server


def test_per_request_cloud_forces_dispatch_on_local_server(tmp_path, monkeypatch):
    """Server default is LOCAL (compute_target=''), but a per-request compute='cloud' forces a
Cloud
    Run dispatch when the control plane is wired (CLOUD_RUN_JOB + GOOGLE_CLOUD_PROJECT + a
bucket)."""
    db = Database(str(tmp_path / "t.db"))  # noqa: F841
    monkeypatch.setattr(m, "db_instance", db)
    bucket = tmp_path / "bucket"
    bucket.mkdir()
    cfg = AppConfig.model_validate(
        {  # noqa: F841
            "deployment": {"compute_target": ""},  # server default = in-process
            "storage": {"backend": "local", "output_root": str(bucket)},
        }
    )
```

```
    monkeypatch.setattr(m, "global_config", cfg)
    monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
    fake_local = tmp_path / "fetched.mp4"
    fake_local.write_bytes(b"FAKEVIDEOBYTES")
    monkeypatch.setattr(
        m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
    )
    monkeypatch.setattr(
        m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
    )
    monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
    monkeypatch.setenv("GOOGLE_CLOUD_PROJECT", "proj")
    fake = _FakeCloudClient()
    _inject_fake(monkeypatch, fake)

    out = m._execute_processing(
        m.global_config,
        m.db_instance,
        m.ProcessRequest(
            video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="cloud"
        ),
    )
    assert out["status"] == "success"
    assert len(out["play_segments"]) == 2
    assert (
        len(fake.calls) == 1
    )  # per-request override dispatched despite the LOCAL server default


def test_per_request_cloud_unconfigured_fails_clearly(tmp_path, monkeypatch):
    """compute='cloud' on a server NOT wired for it fails clearly (no silent local fallback, no
    dispatch) so the picker can surface 'cloud isn't available here'."""
    db = Database(str(tmp_path / "t.db"))  # noqa: F841
    monkeypatch.setattr(m, "db_instance", db)
    cfg = AppConfig.model_validate(
        {"deployment": {"compute_target": ""}}
    )  # no bucket, no env  # noqa: F841
    monkeypatch.setattr(m, "global_config", cfg)
    monkeypatch.setattr(
        m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
    )
    monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
    monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
    video = tmp_path / "v.mp4"
    video.write_bytes(b"hashable-bytes")

    out = m._execute_processing(
        m.global_config,
        m.db_instance,
        m.ProcessRequest(
            video_path=str(video), segmenter="wasb_hybrid", compute="cloud"
        ),
    )
    assert out["status"] == "failed"
    assert "isn't configured" in out["message"]


def test_unknown_compute_value_falls_back_to_server_default(cloud_env, caplog):
    """An unrecognised compute value is ignored (logged) and the server default applies ? here a
    cloud_run server still dispatches to the cloud."""
    db, _bucket, monkeypatch = cloud_env
    fake = _FakeCloudClient()
    _inject_fake(monkeypatch, fake)
    with caplog.at_level("WARNING", logger="backend.main"):
        out = m._execute_processing(
```

```
            m.global_config,
            m.db_instance,
            m.ProcessRequest(
                video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="banana"
            ),
        )
    assert out["status"] == "success"
    assert len(fake.calls) == 1  # fell back to the configured cloud_run default
    assert any("unknown compute" in r.getMessage() for r in caplog.records)


def test_capabilities_reports_cloud_available(cloud_env):
    """A cloud_run-configured server advertises deployment.cloud_available=True (the SPA shows
the
    cloud toggle only then)."""
    _db, _bucket, _monkeypatch = cloud_env
    app = m.create_app(m.global_config, m.db_instance)
    client = TestClient(app)
    resp = client.get("/api/capabilities")
    caps = resp.json()
    assert caps["deployment"]["cloud_available"] is True
```

37. user (2026-07-02T04:52:06.968Z)

```
        len(fake.calls) == 1
    )  # 'cloud' honoured across the boundary ? dispatched despite local default
    segs = db.query_play_segments(job["video_id"], {})
    assert len(segs) == 2
    assert all(_meta(s)["source"] == "cloud_run" for s in segs)
```

--- compute-path PROVENANCE (the validation method: prove which backend ran) ----

```
def test_cloud_dispatch_stamps_compute_provenance(cloud_env):
    """A cloud run stamps result['compute'] with path=cloud_run + the REAL Cloud Run execution
name
    (the GCP-verifiable proof) + the requested choice + job/region/outputs ? so the intended
path is
    provable from the job result alone (no guessing from segment metadata)."""
    _db, _bucket, monkeypatch = cloud_env
    _inject_fake(monkeypatch, _FakeCloudClient())

    out = m._execute_processing(
        m.ProcessRequest(
            video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="cloud"
        )
    )
    prov = out["compute"]
    assert prov["path"] == "cloud_run"
    assert prov["requested"] == "cloud"
    assert (
        prov["execution"] == "exec-1"
    )  # the fake client's execution name (real one in prod)
    assert prov["outputs_uri"].endswith("/outputs/vidCloud/")
    assert prov["job"] and prov["region"]  # the dispatch coordinates are recorded


def test_local_override_stamps_in_process_provenance(cloud_env):
    """On a cloud server, compute='local' runs in-process AND the result proves it:
path=in_process,
    requested=local ? so a 'ran local despite a cloud server' is auditable (no silent
ambiguity)."""
    _db, _bucket, monkeypatch = cloud_env
```

```
    _inject_fake(monkeypatch, _FakeCloudClient())
    monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)

    out = m._execute_processing(
        m.ProcessRequest(
            video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
        )
    )
    assert out["status"] == "success"
    assert out["compute"] == {"path": "in_process", "requested": "local"}


def test_default_in_process_stamps_provenance(tmp_path, monkeypatch):
    """A default-OFF (no cloud) run stamps path=in_process, requested=None ? the byte-identical
local
    path is now self-describing."""
    db = Database(str(tmp_path / "t.db"))  # noqa: F841
    monkeypatch.setattr(m, "db_instance", db)
    monkeypatch.setattr(m, "db_instance", db)
    monkeypatch.setattr(m, "global_config", AppConfig())
    m.app.state.config = AppConfig()
    monkeypatch.setattr(
        m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
    )
    monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
    video = tmp_path / "v.mp4"
    video.write_bytes(b"hashable-bytes")

    out = m._execute_processing(
        m.ProcessRequest(video_path=str(video), segmenter="motion")
    )
    assert out["compute"] == {"path": "in_process", "requested": None}


def test_cloud_requested_unconfigured_stamps_not_dispatched(tmp_path, monkeypatch):
    """compute='cloud' on a server that can't dispatch fails clearly AND records it: target
cloud_run,
    requested=cloud, dispatched=False ? proving the cloud job never ran (no silent local
fallback)."""
    db = Database(str(tmp_path / "t.db"))  # noqa: F841
    monkeypatch.setattr(m, "db_instance", db)
    monkeypatch.setattr(m, "db_instance", db)
    monkeypatch.setattr(
        m,
        "global_config",
        AppConfig.model_validate({"deployment": {"compute_target": ""}}),
    )
    m.app.state.config = AppConfig.model_validate(
        {"deployment": {"compute_target": ""}}
    )
    monkeypatch.setattr(
        m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
    )
    monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
    monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
    video = tmp_path / "v.mp4"
    video.write_bytes(b"hashable-bytes")

    out = m._execute_processing(
        m.ProcessRequest(
            video_path=str(video), segmenter="wasb_hybrid", compute="cloud"
        )
    )
    assert out["status"] == "failed"
    # Honest: it ran NOWHERE (pre-dispatch reject) ? path=not_run, but the intent is recorded.
```

```
    assert out["compute"] == {
        "path": "not_run",
        "requested": "cloud",
        "target": "cloud_run",
        "dispatched": False,
    }


def test_validation_failure_stamps_not_run_provenance(tmp_path, monkeypatch):
    """A validation failure never reaches a segmenter ? path=not_run (nothing executed)."""
    db = Database(str(tmp_path / "t.db"))  # noqa: F841
    monkeypatch.setattr(m, "db_instance", db)
    monkeypatch.setattr(m, "global_config", AppConfig())
    m.app.state.config = AppConfig()
    fail = [
        ValidationResult(validator_name="x", passed=False, message="bad", details={})
    ]
    monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (fail, {}))
    video = tmp_path / "v.mp4"
    video.write_bytes(b"hashable-bytes")

    out = m._execute_processing(m.ProcessRequest(video_path=str(video)))
    assert out["status"] == "failed"
    assert out["compute"]["path"] == "not_run"


def test_full_api_process_job_result_carries_cloud_provenance(cloud_env):
    """End-to-end: POST /api/process (compute=cloud) ? the job result the SPA polls carries the
    cloud_run provenance + execution, so the UI/operator can PROVE the cloud path was taken."""
    _db, _bucket, monkeypatch = cloud_env
    _inject_fake(monkeypatch, _FakeCloudClient(video_id="vidProv"))
    app = m.create_app(m.global_config, _db)
    client = TestClient(app)
```

38. assistant (2026-07-02T04:52:58.349Z)

Let me check `_add_column_idempotent` and grep for other tests pinning the schema version.

39. user (2026-07-02T04:52:59.950Z)

```
78:    def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
79-        """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.
80-
81-        A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn:``
82-        transaction, so an interrupted v3/v4 block can leave the column added while
83-        ``user_version`` stays un-bumped ? and the next open would re-run the same
84-        ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
85-        future ``Database(...)`` construction (``_init_db`` runs in ``__init__``). Swallow
86-        ONLY that specific error so the ladder stays crash-safe, matching the re-runnable
87-        ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
88-        """
89-        try:
90-            cursor.execute(ddl)
91-        except sqlite3.OperationalError as exc:
92-            if "duplicate column name" not in str(exc).lower():
--
294:        self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
295:        self._add_column_idempotent(
296-            cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP"
297-        )
298-        cursor.execute(
299-            "CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status, next_poll_at)"
300-        )
301-        cursor.execute("PRAGMA user_version = 3")
302-
```

```
303-    def _migrate_to_v4(self, cursor):
304-        # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
305-        # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
306-        # path can scope rows to an owner. **NULL = local install = byte-identical to
307-        # today** ? every existing row stays NULL and every existing query (which never

[Showing results with pagination = limit: 30]
```

40. user (2026-07-02T04:53:01.345Z)

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_async_jobs.py:10:- migration: fresh DB lands at user_version 2 with a jobs
table; v1 DBs upgrade in place
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_async_jobs.py:98:        assert conn.execute("PRAGMA
user_version").fetchone()[0] == 9
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_async_jobs.py:132:    conn.execute("PRAGMA user_version = 1")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_async_jobs.py:138:        assert c.execute("PRAGMA
user_version").fetchone()[0] == 9
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_billing.py:47:        ver = conn.execute("PRAGMA user_version").fetchone()[0]
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_database.py:102:    version = conn.execute("PRAGMA
user_version").fetchone()[0]
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_database.py:131:    version = conn.execute("PRAGMA
user_version").fetchone()[0]
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_database.py:270:    conn.execute("PRAGMA user_version = 3")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_database.py:278:    assert conn.execute("PRAGMA user_version").fetchone()[0]
== 9
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_database.py:302:    on its own, so a crash after the ADD COLUMNs but before
the ``PRAGMA user_version``
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_database.py:312:    # but the process died before the version bump, so
user_version is stuck at 2.
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_database.py:315:    conn.execute("PRAGMA user_version = 2")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_database.py:325:    assert conn.execute("PRAGMA user_version").fetchone()[0]
== 9
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_database.py:417:def test_upsert_is_idempotent_on_user_version(tmp_path):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-
process\tests\test_db_upgrade_safety.py:11:def _user_version(path: str) -> int:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
```

indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_db_upgrade_safety.py:14:   return int(conn.execute("PRAGMA user_version").fetchone()[0])
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_db_upgrade_safety.py:19:def _set_user_version(path: str, v: int) -> None:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_db_upgrade_safety.py:22:   conn.execute(f"PRAGMA user_version = {int(v)}")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_db_upgrade_safety.py:33:   assert _user_version(p) == SCHEMA_VERSION
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_db_upgrade_safety.py:53:   _set_user_version(p, SCHEMA_VERSION + 5) # a DB from a hypothetical newer app
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_db_upgrade_safety.py:62:   _set_user_version(p, 3) # simulate an older on-disk schema
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_db_upgrade_safety.py:67:   assert _user_version(p) == SCHEMA_VERSION # upgraded
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_db_upgrade_safety.py:68:   assert _user_version(str(backup)) == 3 # the backup is the PRE-upgrade snapshot
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_db_upgrade_safety.py:81:   assert _user_version(dest) == SCHEMA_VERSION
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_job_worker_loop.py:300:   assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_job_worker_loop.py:304:   # A pre-migration legacy DB (only `videos`, user_version 0) must run the FULL ordered
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_job_worker_loop.py:313:   conn.execute("PRAGMA user_version = 0")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_job_worker_loop.py:318:   assert c.execute("PRAGMA user_version").fetchone()[0] == 9
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_locale_analytics.py:23:def _user_version(path):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_locale_analytics.py:26:   return conn.execute("PRAGMA user_version").fetchone()[0]
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_locale_analytics.py:34:   assert _user_version(path) == 9 # head: v7 added locale, v8 compute, v9 compile job
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_locale_analytics.py:45:   conn.execute("PRAGMA user_version = 6")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-idempotent-process\tests\test_locale_analytics.py:51:   assert _user_version(path) == 9

41. user (2026-07-02T04:53:15.314Z)

```
        assert by_name[col][3] == 1, f"{col} should be NOT NULL"


def test_schema_version_is_current(tmp_path):
    db_path = tmp_path / "test.db"
    Database(str(db_path))
    conn = sqlite3.connect(str(db_path))
    version = conn.execute("PRAGMA user_version").fetchone()[0]
    conn.close()
    # v9 = async compile job (kind/params); v8 = compute-picker; v7 = job UI locale; v6 = cost
ledger.
    # Bump with each migration.
    assert version == 9


def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
    db_path = tmp_path / "legacy.db"
    # Simulate a pre-v1 database that only has the old tables and no version.
    conn = sqlite3.connect(str(db_path))
    conn.execute(
        "CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT, status TEXT)"
    )
    conn.commit()
    conn.close()

    # Opening with the new Database should add candidate_segments without error...
    Database(str(db_path))
    # ...and running init again must be a no-op (idempotent).
    Database(str(db_path))

    conn = sqlite3.connect(str(db_path))
    tables = [
        r[0]
        for r in conn.execute(
            "SELECT name FROM sqlite_master WHERE type='table'"
        ).fetchall()
    ]
    version = conn.execute("PRAGMA user_version").fetchone()[0]
    conn.close()
    assert "candidate_segments" in tables
    assert "jobs" in tables  # v2 (async job model, #16)
    assert "user_models" in tables  # v5 (per-user model store)
----260-330----
        "vidA", "wasb_hybrid", 2.0, 9.0, stats={"net_crossings": 3}
    )
    conn = sqlite3.connect(str(db_path))
    conn.execute("DROP INDEX IF EXISTS idx_videos_user")
    conn.execute("DROP INDEX IF EXISTS idx_candidates_user")
    conn.execute("DROP TABLE IF EXISTS user_models")
    # Recreate v3-shaped tables WITHOUT user_id by rebuilding (simplest: just rewind version
    # and drop the v4/v5 artifacts that a true v3 DB wouldn't have).
    conn.execute("ALTER TABLE videos DROP COLUMN user_id")
    conn.execute("ALTER TABLE candidate_segments DROP COLUMN user_id")
    conn.execute("PRAGMA user_version = 3")
    conn.commit()
    conn.close()
    assert "user_id" not in _table_cols(db_path, "videos")

    # Re-open: the migration ladder must carry the v3 DB up to the head (v9) without data loss.
    db2 = Database(str(db_path))
    conn = sqlite3.connect(str(db_path))
    assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
    tables = {
```

```
        r[0]
        for r in conn.execute(
            "SELECT name FROM sqlite_master WHERE type='table'"
        ).fetchall()
    }
    assert "user_models" in tables  # v5
    assert "pricebook" in tables  # v6 (cost ledger)
    conn.close()
    assert "user_id" in _table_cols(db_path, "videos")
    assert "cost_usd" in _table_cols(
        db_path, "jobs"
    )  # v6 cost columns back-filled (NULL)
    assert "user_id" in _table_cols(db_path, "candidate_segments")

    v = db2.get_video("vidA")
    assert v is not None and v["validation_results"] == [{"v": 1}]
    cands = db2.query_candidate_segments("vidA")
    assert len(cands) == 1 and cands[0]["stats"]["net_crossings"] == 3


def test_migration_idempotent_on_interrupted_upgrade(tmp_path):
    """An INTERRUPTED v3/v4 upgrade must not brick the DB. A DDL ``ALTER`` auto-commits
    on its own, so a crash after the ADD COLUMNs but before the ``PRAGMA user_version``
    bump leaves the columns present while the version is stale ? re-opening re-runs the
    bare ALTERs. The ladder must tolerate that (re-runnable, like the v1/v2/v5 blocks)
    instead of raising ``OperationalError: duplicate column name``."""
    db_path = tmp_path / "interrupted.db"
    Database(
        str(db_path)
    )  # fully migrate to the current version (v3/v4 columns all present)

    # Simulate the interruption: the v3/v4 ADD COLUMNs already landed and auto-committed,
    # but the process died before the version bump, so user_version is stuck at 2.
    conn = sqlite3.connect(str(db_path))
    conn.execute("DROP TABLE IF EXISTS user_models")  # a true v2 DB lacks the v5 table
    conn.execute("PRAGMA user_version = 2")
    conn.commit()
    conn.close()

    # Re-opening re-runs v3 (jobs.policy/next_poll_at), v4 (videos/candidate_segments.user_id),
    # v5 (user_models), v6 (jobs cost columns + pricebook), v7 (jobs.locale), v8 (jobs.compute)
and
    # v9 (jobs.kind/params) over artifacts that ALREADY exist ? must NOT raise (re-runnable
ALTERs /
    # INSERT OR IGNORE), and must reach v9.
    Database(str(db_path))
    conn = sqlite3.connect(str(db_path))
    assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
    conn.close()
    # Columns/tables are all present after the clean re-apply.
    assert "policy" in _table_cols(db_path, "jobs")
    assert "cost_usd" in _table_cols(db_path, "jobs")  # v6 re-applied cleanly
    assert "user_id" in _table_cols(db_path, "videos")
----billing----
```

---------- v6 migration + pricebook seed

```
def test_v6_migration_adds_cost_columns_and_seeds_placeholder(tmp_path):
    db = Database(str(tmp_path / "m.db"))
    with db._connect() as conn:
        ver = conn.execute("PRAGMA user_version").fetchone()[0]
        cols = {r[1] for r in conn.execute("PRAGMA table_info(jobs)").fetchall()}
        seeded = conn.execute(
```

```
            "SELECT unit_price_usd FROM pricebook WHERE version='placeholder-v0'"
        ).fetchall()
    assert ver >= 6
    assert {
        "cost_usd",
        "gpu_active_seconds",
----loop----
    # status='waiting' is a value, not a column; assert the round-trip works.
    db.create_job("j", "/v.mp4")
    db.set_job_waiting("j", delay_sec=5)
    assert db.get_job("j")["status"] == "waiting"


def test_fresh_db_is_v9(tmp_path):
    db = Database(str(tmp_path / "v.db"))
    with db._get_connection() as conn:
        # Head of the migration ladder (bump when database.py adds a new `if version < N`).
        assert conn.execute("PRAGMA user_version").fetchone()[0] == 9


def test_legacy_db_upgrades_to_head(tmp_path):
    # A pre-migration legacy DB (only `videos`, user_version 0) must run the FULL ordered
    # migration ladder up to the current head (v9) without error ? this exercises the v0?v9
    # path including the v1 `candidate_segments` CREATE that the later v4 ALTER depends on.
    path = str(tmp_path / "legacy.db")
    conn = sqlite3.connect(path)
    conn.execute(
        "CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT NOT NULL, "
        "processed_at TIMESTAMP, status TEXT NOT NULL, validation_results TEXT)"
    )
    conn.execute("PRAGMA user_version = 0")
    conn.commit()
    conn.close()
    db = Database(path)
    with db._get_connection() as c:
        assert c.execute("PRAGMA user_version").fetchone()[0] == 9
----locale----
        conn.close()


def _user_version(path):
    conn = sqlite3.connect(path)
    try:
        return conn.execute("PRAGMA user_version").fetchone()[0]
    finally:
        conn.close()


def test_fresh_db_v7_has_jobs_locale_column(tmp_path):
    path = str(tmp_path / "fresh.db")
    Database(path)
    assert _user_version(path) == 9  # head: v7 added locale, v8 compute, v9 compile job
    assert "locale" in _cols(path, "jobs")


def test_v6_db_upgrades_to_v7_and_backfills_locale_null(tmp_path):
    path = str(tmp_path / "v6.db")
    db = Database(path)  # born at v7
    db.create_job("j-old", "/v.mp4")  # a prior row, no locale
    # Simulate a real v6 jobs table (no locale column, version pinned at 6).
    conn = sqlite3.connect(path)
    conn.execute("ALTER TABLE jobs DROP COLUMN locale")
    conn.execute("PRAGMA user_version = 6")
    conn.commit()
    conn.close()
```

```
    assert "locale" not in _cols(path, "jobs")

    Database(path)  # re-open ? the ladder runs _migrate_to_v7 (+ v8, v9) up to the head
    assert _user_version(path) == 9
    assert "locale" in _cols(path, "jobs")
    # The prior row survived the ALTER and back-filled to NULL (not dropped, not defaulted to a
value).
    conn = sqlite3.connect(path)
    try:
        assert (
            conn.execute("SELECT locale FROM jobs WHERE id='j-old'").fetchone()[0]
            is None
        )
    finally:
```

42. assistant (2026-07-02T04:53:22.029Z)

I now have the full picture. Let me implement. Starting with the DB layer (migration v10, mirroring v8's `compute`).

43. user (2026-07-02T04:53:25.020Z)

```
10
11      logger = logging.getLogger(__name__)
12
13      # The highest schema version this binary knows how to migrate TO (the top of the
migration ladder
14      # in `_init_db`). PACKAGING_PLAN.md P2d uses it for the upgrade-safety contract on a
downloadable,
15      # auto-updating app: refuse to open a DB from a NEWER app (downgrade guard) and back the
DB up before
16      # an upgrade migration runs.
`tests/test_database.py::test_schema_version_matches_ladder` pins it to
17      # the ladder so adding a `_migrate_to_vN` without bumping this fails CI.
18      SCHEMA_VERSION = 9
19
20
21      class SchemaTooNewError(RuntimeError):
22          """The on-disk DB was created by a newer app version than this binary understands
(P2d)."""
23
24
```

44. user (2026-07-02T04:53:33.651Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

45. user (2026-07-02T04:53:36.004Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

46. user (2026-07-02T04:53:42.344Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

47. user (2026-07-02T04:53:56.770Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

48. assistant (2026-07-02T04:54:01.623Z)

Now `backend/main.py` ? the `ProcessRequest.force` field, the short-circuit, and the endpoint threading:

49. user (2026-07-02T04:54:06.568Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/backend/main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

50. user (2026-07-02T04:54:13.079Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/backend/main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

51. user (2026-07-02T04:54:33.044Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/backend/main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

52. user (2026-07-02T04:54:42.694Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/backend/main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

53. user (2026-07-02T04:54:47.563Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/backend/api/job_worker.py has been updated successfully. (file state is current in your context ? no need to Read it back)

54. assistant (2026-07-02T04:54:57.600Z)

Now update the existing tests that pin the schema version literal, plus the crash-restore test that now needs `force=true`:

55. user (2026-07-02T04:55:01.864Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/tests/test_async_jobs.py has been updated successfully. (file state is current in your context ? no need to Read it back)

56. user (2026-07-02T04:55:04.758Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/tests/test_async_jobs.py has been updated successfully. (file state is current in your context ? no need to Read it back)

57. user (2026-07-02T04:55:07.515Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/tests/test_async_jobs.py has been updated successfully. (file state is current in your context ? no need to Read it back)

58. user (2026-07-02T04:55:11.775Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/tests/test_async_jobs.py has been updated successfully. (file state is current in your context ? no need to Read it back)

59. user (2026-07-02T04:55:24.706Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

60. user (2026-07-02T04:55:25.777Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

61. user (2026-07-02T04:55:28.329Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

62. user (2026-07-02T04:55:34.893Z)

```
95              assert by_name[col][3] == 1, f"{col} should be NOT NULL"
96
97
98      def test_schema_version_is_current(tmp_path):
99          db_path = tmp_path / "test.db"
100         Database(str(db_path))
101         conn = sqlite3.connect(str(db_path))
102         version = conn.execute("PRAGMA user_version").fetchone()[0]
103         conn.close()
104         # v9 = async compile job (kind/params); v8 = compute-picker; v7 = job UI locale; v6
= cost ledger.
105         # Bump with each migration.
106         assert version == 9
107
108
109     def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
```

63. user (2026-07-02T04:55:43.758Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/tests/test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

64. user (2026-07-02T04:55:47.085Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/tests/test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

65. user (2026-07-02T04:55:54.219Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/tests/test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

66. assistant (2026-07-02T04:55:58.223Z)

Now `test_job_worker_loop.py` and `test_locale_analytics.py`:

67. user (2026-07-02T04:55:59.966Z)

```
294
295
296     def test_fresh_db_is_v9(tmp_path):
297         db = Database(str(tmp_path / "v.db"))
298         with db._get_connection() as conn:
299             # Head of the migration ladder (bump when database.py adds a new `if version <
N`).
300             assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
301
302
303     def test_legacy_db_upgrades_to_head(tmp_path):
304         # A pre-migration legacy DB (only `videos`, user_version 0) must run the FULL
ordered
305         # migration ladder up to the current head (v9) without error ? this exercises the
v0?v9
306         # path including the v1 `candidate_segments` CREATE that the later v4 ALTER depends
on.
307         path = str(tmp_path / "legacy.db")
308         conn = sqlite3.connect(path)
309         conn.execute(
310             "CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT NOT NULL, "
311             "processed_at TIMESTAMP, status TEXT NOT NULL, validation_results TEXT)"
312         )
313         conn.execute("PRAGMA user_version = 0")
314         conn.commit()
315         conn.close()
316         db = Database(path)
317         with db._get_connection() as c:
318             assert c.execute("PRAGMA user_version").fetchone()[0] == 9
319
```

68. user (2026-07-02T04:56:16.011Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-
process/tests/test_job_worker_loop.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

69. user (2026-07-02T04:56:18.118Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-
process/tests/test_job_worker_loop.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

70. user (2026-07-02T04:56:25.541Z)

```
30
31      def test_fresh_db_v7_has_jobs_locale_column(tmp_path):
32          path = str(tmp_path / "fresh.db")
33          Database(path)
34          assert _user_version(path) == 9  # head: v7 added locale, v8 compute, v9 compile job
35          assert "locale" in _cols(path, "jobs")
36
37
38      def test_v6_db_upgrades_to_v7_and_backfills_locale_null(tmp_path):
39          path = str(tmp_path / "v6.db")
40          db = Database(path)  # born at v7
```

```
41          db.create_job("j-old", "/v.mp4")  # a prior row, no locale
42          # Simulate a real v6 jobs table (no locale column, version pinned at 6).
43          conn = sqlite3.connect(path)
44          conn.execute("ALTER TABLE jobs DROP COLUMN locale")
45          conn.execute("PRAGMA user_version = 6")
46          conn.commit()
47          conn.close()
48          assert "locale" not in _cols(path, "jobs")
49
50          Database(path)  # re-open ? the ladder runs _migrate_to_v7 (+ v8, v9) up to the head
51          assert _user_version(path) == 9
52          assert "locale" in _cols(path, "jobs")
53          # The prior row survived the ALTER and back-filled to NULL (not dropped, not
defaulted to a value).
54          conn = sqlite3.connect(path)
```

71. user (2026-07-02T04:56:40.915Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/tests/test_locale_analytics.py has been updated successfully. (file state is current in your context ? no need to Read it back)

72. user (2026-07-02T04:56:41.389Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process/tests/test_locale_analytics.py has been updated successfully. (file state is current in your context ? no need to Read it back)

73. assistant (2026-07-02T04:57:18.207Z)

Now let me verify `get_job`/`claim_due_job` return full rows (so `force` survives the claim):

74. user (2026-07-02T04:57:19.867Z)

```
1100                            "  (julianday(next_poll_at) - julianday('now')) * 86400.0 END)
AS secs "
1101                            "FROM jobs WHERE status='waiting'"
1102                        )
1103                        .fetchone()
1104                    )
1105                if row is None or row["secs"] is None:
1106                    return None
1107                return max(0.0, float(row["secs"]))
1108            except Exception as e:
1109                logger.error(f"Failed to compute next-due delay: {e}")
1110                return None
1111
1112        def claim_due_job(self) -> Optional[Dict[str, Any]]:
1113            """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
1114            flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
1115            double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
1116            first. Returns the claimed job dict, or None if nothing is runnable."""
1117            try:
1118                with self._connect() as conn:
1119                    cur = conn.cursor()
1120                    row = cur.execute(
1121                        "SELECT id, status FROM jobs WHERE status='queued' "
1122                        "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
```

```
                        "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT 1"
                    ).fetchone()
                    if not row:
                        return None
                    cur.execute(
                        "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
                        "WHERE id=? AND status=?",
                        (row["id"], row["status"]),
                    )
                    conn.commit()
                    if cur.rowcount != 1:
                        return None  # lost the race to a concurrent worker
                return self.get_job(row["id"])
            except Exception as e:
                logger.error(f"Failed to claim a due job: {e}")
                return None

        def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
            """Crash recovery: any job still queued/running from a previous process is dead
            (the executor is in-process). Mark failed so clients see a terminal state, not a
            hang. Called once at startup. Returns the number of jobs swept."""
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    cur.execute(
                        "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
                        "WHERE status IN ('queued', 'running')",
                        (reason,),
                    )
                    conn.commit()
                    return cur.rowcount
            except Exception as e:
                logger.error(f"Failed to sweep stale jobs: {e}")
                return 0

        # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) ------------------------
        # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
        # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
        # serving.py) resolves them, and wiring that bridge into the production decision
seam
        # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
        # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
        # JSON-serialised dicts; the rest are scalar columns.

        def upsert_user_model(
            self,
            user_id: Optional[str],
            *,
            version: int = 1,
            baseline_version: Optional[str] = None,
            feature_schema_hash: Optional[str] = None,
            fusion_config: Optional[Dict[str, Any]] = None,
            classifier_json: Optional[Dict[str, Any]] = None,
            promotion_evidence: Optional[Dict[str, Any]] = None,
            n_videos_at_fit: int = 0,
            is_active: bool = False,
        ) -> Optional[int]:
            """Insert or replace ONE per-user model row keyed by ``(user_id, version)``.
```

75. user (2026-07-02T04:57:27.511Z)

```
901:    def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
902-        """Fetch one job; `result`/`player_pool` JSON deserialized."""
903-        with self._connect() as conn:
```

```
904-            row = (
905-                conn.cursor()
906-                .execute("SELECT * FROM jobs WHERE id = ?", (job_id,))
907-                .fetchone()
908-            )
909-            if not row:
910-                return None
911-            d = dict(row)
912-            d["result"] = json.loads(d["result"]) if d["result"] else None
913-            d["player_pool"] = (
914-                json.loads(d["player_pool"]) if d["player_pool"] else None
915-            )
916-            d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
917-            return d
918-
919-    # --- Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8)
----------------------------
920-    def get_pricebook_rates(
921-        self, version: str, gpu_type: Optional[str] = None
922-    ) -> Dict[str, float]:
923-        """Resolve a flat ``{resource: unit_price_usd}`` for a pricebook ``version``. The
per-second
```

76. assistant (2026-07-02T04:57:34.614Z)

`SELECT *` ? force survives the claim. Now the new test file:

77. assistant (2026-07-02T04:58:14.152Z)

```
"""Idempotent /api/process by content hash (#340; reuse mechanism also tracked as #330).

Contract under test:
- Re-submitting an already-processed video (same file ? same MD5 video_id, status
  'processed', stored play_segments) returns the STORED segments as the job result with
  ZERO segmenter invocations (no Gemini call) and ZERO compute dispatch, logging a clear
  '[IDEMPOTENT]' line. compute provenance stamps path='not_run' (nothing executed).
- `force=true` is the explicit opt-in to reprocess (the destroy-after-confirm re-ingest
  path) and must survive the async submit?worker boundary like v8's `compute`
  (jobs.force, schema v10).
- The short-circuit requires status == 'processed' AND stored segments: an unprocessed
  video, a prior 'failed' record, or a processed-but-segmentless video still re-runs.
"""

import pytest

pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
from fastapi.testclient import TestClient

import backend.main as m
from backend.config.models import AppConfig
from backend.infrastructure.database import Database
from backend.pipeline.validators.base import ValidationResult


class _InlineExecutor:
    """submit() runs the fn synchronously ? deterministic tests, no threads."""

    def submit(self, fn, *args, **kwargs):
        fn(*args, **kwargs)


@pytest.fixture
def env(tmp_path, monkeypatch):
    db = Database(str(tmp_path / "t.db"))
    cfg = AppConfig()
```

```
    monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
    app = m.create_app(cfg, db)
    client = TestClient(app)
    return client, db, tmp_path, monkeypatch


def _video(tmp_path, name="m.mp4", content=b"not-a-real-video-but-hashable"):
    p = tmp_path / name
    p.write_bytes(content)
    return p


def _pass():
    return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]


def _seg_factory(calls, start=0.0, end=5.0):
    """A fake segmenter CLASS that records every process_video call in ``calls`` ?
    the 'no Gemini call' assertion is `calls` staying empty/unchanged."""

    class _Seg:
        def __init__(self, db, cfg):
            self.db = db

        def process_video(self, video_id, video_path, *a, **k):
            calls.append(video_id)
            sid = self.db.add_play_segment(video_id, start, end)
            return [
                {
                    "id": sid,
                    "start_time": start,
                    "end_time": end,
                    "duration": end - start,
                    "metadata": {},
                }
            ]

    return _Seg


def _submit(client, vid_file, **extra):
    body = {"video_path": str(vid_file), "segmenter": "x", **extra}
    r = client.post("/api/process", json=body)
    assert r.status_code == 202
    return client.get(f"/api/jobs/{r.json()['job_id']}").json()
```

--- the short-circuit hit ----------

```
def test_reprocess_short_circuits_with_no_segmenter_call(env, caplog):
    """#340 acceptance: the re-submit returns the cached rallies with ZERO segmenter
    (Gemini) calls and ZERO compute dispatch, and says so ('[IDEMPOTENT]' log)."""
    client, db, tmp_path, mp = env
    vid_file = _video(tmp_path)
    vid = m.compute_video_id(str(vid_file))
    calls: list = []
    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
    mp.setattr(m.segmenter_registry, "get", lambda name: _seg_factory(calls))
    dispatches: list = []
    real_dispatch = m._maybe_dispatch_remote

    def _dispatch_spy(*a, **k):
        dispatches.append(a)
```

```
        return real_dispatch(*a, **k)

    mp.setattr(m, "_maybe_dispatch_remote", _dispatch_spy)

    # First run: the segmenter genuinely runs once.
    first = _submit(client, vid_file)
    assert first["result"]["status"] == "success"
    assert calls == [vid]
    assert len(dispatches) == 1

    # Re-submit (a refresh / SPA "Try again"): stored result, NO new segmenter call,
    # NO new dispatch attempt.
    with caplog.at_level("INFO", logger="backend.main"):
        second = _submit(client, vid_file)
    assert calls == [vid], "re-submit must NOT invoke the segmenter again"
    assert len(dispatches) == 1, "re-submit must NOT reach compute dispatch"
    assert second["status"] == "done"
    result = second["result"]
    assert result["status"] == "success"
    assert result["idempotent"] is True
    assert len(result["play_segments"]) == 1
    assert result["play_segments"][0]["start_time"] == 0.0
    # Provenance: nothing executed on the short-circuit.
    assert result["compute"]["path"] == "not_run"
    assert any("[IDEMPOTENT]" in r.getMessage() for r in caplog.records)
    # The stored row is untouched (same single segment).
    segs = db.query_play_segments(vid, {})
    assert len(segs) == 1 and segs[0]["start_time"] == 0.0
```

--- force=true bypass ----------

```
def test_force_true_reprocesses_and_replaces_segments(env):
    """force=true re-runs the segmenter (explicit opt-in) under the existing
    destroy-after-confirm re-ingest semantics: the new run's segments replace the old."""
    client, db, tmp_path, mp = env
    vid_file = _video(tmp_path)
    vid = m.compute_video_id(str(vid_file))
    first_calls: list = []
    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
    mp.setattr(m.segmenter_registry, "get", lambda name: _seg_factory(first_calls))
    assert _submit(client, vid_file)["result"]["status"] == "success"
    assert first_calls == [vid]

    # Deliberate reprocess with a run that yields DIFFERENT segments.
    forced_calls: list = []
    mp.setattr(
        m.segmenter_registry,
        "get",
        lambda name: _seg_factory(forced_calls, start=10.0, end=12.0),
    )
    body = _submit(client, vid_file, force=True)
    assert forced_calls == [vid], "force=true must re-run the segmenter"
    result = body["result"]
    assert result["status"] == "success"
    assert "idempotent" not in result
    # Destroy-after-confirm: the fresh run replaced the prior segments.
    segs = db.query_play_segments(vid, {})
    assert len(segs) == 1 and segs[0]["start_time"] == 10.0
```

--- non-hits: unprocessed / failed / segmentless videos still run ----------

```
def test_unprocessed_video_runs_segmenter_normally(env):
    client, db, tmp_path, mp = env
    vid_file = _video(tmp_path)
    vid = m.compute_video_id(str(vid_file))
    calls: list = []
    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
    mp.setattr(m.segmenter_registry, "get", lambda name: _seg_factory(calls))
    body = _submit(client, vid_file)
    assert calls == [vid]
    assert body["result"]["status"] == "success"
    assert "idempotent" not in body["result"]


def test_prior_failed_record_still_reprocesses(env):
    """A video whose stored status is 'failed' must NOT short-circuit ? even if prior
    good segments survived a failed re-validation (the upsert keeps them)."""
    client, db, tmp_path, mp = env
    vid_file = _video(tmp_path)
    vid = m.compute_video_id(str(vid_file))
    db.add_video(vid, str(vid_file), "failed", [])
    db.add_play_segment(vid, 1.0, 2.0)  # prior good result kept by the failed upsert
    calls: list = []
    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
    mp.setattr(m.segmenter_registry, "get", lambda name: _seg_factory(calls))
    body = _submit(client, vid_file)
    assert calls == [vid], "a 'failed' record must re-run the segmenter"
    assert body["result"]["status"] == "success"
    assert db.get_video(vid)["status"] == "processed"


def test_processed_without_stored_segments_reprocesses(env):
    """status='processed' alone is not enough ? the short-circuit also requires stored
    play_segments (an empty prior run has nothing to reuse)."""
    client, db, tmp_path, mp = env
    vid_file = _video(tmp_path)
    vid = m.compute_video_id(str(vid_file))
    db.add_video(vid, str(vid_file), "processed", [])  # processed, but 0 segments
    calls: list = []
    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
    mp.setattr(m.segmenter_registry, "get", lambda name: _seg_factory(calls))
    body = _submit(client, vid_file)
    assert calls == [vid], "processed-but-segmentless must re-run the segmenter"
    assert body["result"]["status"] == "success"
    assert len(db.query_play_segments(vid, {})) == 1
```

--- async boundary threading (mirror of the v8 `compute` round-trip) ----------

```
def test_force_round_trips_the_async_job_boundary(env, tmp_path):
    """v10: force must round-trip submit ? jobs row ? _job_to_request so the worker
    honours it across the async boundary. NULL/0 (pre-v10 rows / older clients) ? False
    = the idempotent default."""
    client, db, tpath, mp = env
    assert m.ProcessRequest(video_path="x").force is False  # default-safe

    db.create_job("j-force", "/v.mp4", force=True)
    db.create_job("j-default", "/v.mp4")
    assert db.get_job("j-force")["force"] == 1
    assert db.get_job("j-default")["force"] == 0
    assert m._job_to_request(db.get_job("j-force")).force is True
    assert m._job_to_request(db.get_job("j-default")).force is False

    # A pre-v10 row (force back-filled to NULL by the ALTER) reconstructs to False.
```

```
    db._execute_write(
        "UPDATE jobs SET force=NULL WHERE id=?", ("j-default",), "test setup"
    )
    assert db.get_job("j-default")["force"] is None
    assert m._job_to_request(db.get_job("j-default")).force is False

    # And the HTTP boundary: a force=true submit persists force=1 on the job row.
    vid_file = _video(tpath, name="b.mp4")
    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
    mp.setattr(m.segmenter_registry, "get", lambda name: _seg_factory([]))
    r = client.post(
        "/api/process",
        json={"video_path": str(vid_file), "segmenter": "x", "force": True},
    )
    assert r.status_code == 202
    assert db.get_job(r.json()["job_id"])["force"] == 1
```

78. user (2026-07-02T04:58:14.192Z)

```
File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-
process/tests/test_process_idempotency.py (file state is current in your context ? no need to
Read it back)
```

79. assistant (2026-07-02T04:58:14.710Z)

```
You've hit your session limit · resets 1:10pm (Asia/Calcutta)
```